

Facultade de Informática
UNIVERSIDADE DA CORUÑA

TRABALLO FIN DE GRAO
GRAO EN ENXEÑARÍA INFORMÁTICA
MENCIÓN EN COMPUTACIÓN



Sistema genérico y extensible para el ajuste fino y uso de modelos generativos

Estudante: David González Torres

Dirección: Daniel Rivero Cebrián

Alejandro Puente Castro

A Coruña, junio de 2026.

A mis padres, mi hermano y mis amigos

Agradecimientos

Quiero darle las gracias a mi madre, a mi padre y a mi hermano por estar siempre ahí, ayudándome, apoyándome y motivándome a superarme cada día.

También al resto de mi familia, tanto a quienes están y se sienten orgullosos como a quienes ya no están y sé que también lo estarían.

A mis amigos, por ayudarme a desconectar y por tantos momentos y experiencias disfrutados con ellos.

Y, por último, agradecer a mis tutores Daniel y Alejandro por haberme ayudado en la elección del tema, durante la realización del proyecto y, sobre todo, a aprender.

Resumen

En los últimos años se ha vuelto muy popular el uso de modelos de inteligencia artificial generativa capaces de generar texto, imágenes, audio o vídeos a partir de diferentes modalidades de entrada. Las técnicas de ajuste fino permiten partir de un modelo genérico preentrenado y especializarlo en un dominio o tarea en concreto, reduciendo así el coste computacional y el tamaño del *dataset* necesario para el entrenamiento en comparación con el entrenamiento desde cero.

Actualmente existen diversas librerías y sistemas que proporcionan soporte para la inferencia y el ajuste fino. Sin embargo, la mayoría de estas herramientas están enfocadas en arquitecturas, modalidades o técnicas en concreto y no integran de manera unificada todas las fases del proceso en un único sistema modular y extensible con interfaz gráfica.

En este contexto, se ha desarrollado un sistema genérico y extensible que integra la carga de modelos y de *datasets*, la inferencia, el procesamiento de entradas y salidas, y el ajuste fino de modelos generativos a través de una interfaz gráfica. El sistema sigue una arquitectura modular que desacopla los diferentes componentes, facilitando la incorporación de nuevas modalidades, arquitecturas y técnicas de ajuste fino.

Para la validación del sistema, se ha implementado la carga de recursos desde Hugging Face y almacenamiento local, soporte para modelos de difusión con arquitecturas como Stable Diffusion y Stable Diffusion XL y las técnicas de ajuste fino de LoRA y DreamBooth.

Abstract

In recent years, the use of generative artificial intelligence models capable of generating text, images, audio or videos from different input modalities has become very popular. Fine-tuning techniques make it possible to start from a generic pre-trained model and specialize it in a specific domain or task, thus reducing the computational cost and the size of the *dataset* required for training compared to training from scratch.

Currently, there are several libraries and systems that provide support for inference and fine-tuning. However, most of these tools are focused on specific architectures, modalities or techniques, and do not integrate all stages of the process in a unified way within a single

modular and extensible system with a graphical interface.

In this context, a generic and extensible system has been developed that integrates model and *dataset* loading, inference, input and output processing, and fine-tuning of generative models through a graphical interface. The system follows a modular architecture that decouples the different components, facilitating the incorporation of new modalities, architectures and fine-tuning techniques.

For the validation of the system, resource loading from Hugging Face and local storage, support for diffusion models with architectures such as Stable Diffusion and Stable Diffusion XL, and the LoRA and DreamBooth fine-tuning techniques have been implemented.

Palabras clave:

- Inteligencia artificial
- Modelos generativos
- Ajuste fino
- Inferencia
- LoRA
- DreamBooth

Keywords:

- Artificial Intelligence
- Generative Models
- Fine-tuning
- Inference
- LoRA
- DreamBooth

Índice general

1 Introducción **1**

1.1 Objetivos . . . 2

1.2 Estructura de la memoria . . . 3

2 Fundamentos teóricos **4**

2.1 Modelos generativos . . . 4

2.2 Ajuste fino . . . 6

2.3 LoRA . . . 6

2.4 DreamBooth . . . 7

3 Materiales y métodos **8**

3.1 Python . . . 8

3.2 Hugging Face . . . 9

3.3 Streamlit . . . 9

4 Estudio de mercado **11**

4.1 Herramientas existentes . . . 11

4.1.1 Herramientas para inferencia . . . 11

4.1.2 Herramientas para ajuste fino . . . 12

4.2 Comparativa . . . 13

4.3 Análisis DAFO . . . 14

4.4 Conclusión . . . 15

5 Planificación del proyecto **17**

5.1 Metodología . . . 17

5.2 Planificación inicial . . . 17

5.3 Calendario final . . . 22

6 Desarrollo del sistema **25**
6.1 Requisitos . . . 25
6.1.1 Requisitos funcionales . . . 25
6.1.2 Requisitos no funcionales . . . 26
6.2 Arquitectura general . . . 26
6.3 Iteración 1: Carga de modelos y *datasets* . . . 27
6.3.1 Análisis . . . 27
6.3.2 Diseño . . . 27
6.3.3 Implementación . . . 33
6.3.4 Pruebas . . . 35
6.4 Iteración 2: Inferencia . . . 36
6.4.1 Análisis . . . 37
6.4.2 Diseño . . . 38
6.4.3 Implementación . . . 41
6.4.4 Pruebas . . . 41
6.5 Iteración 3: Procesamiento de entradas y salidas . . . 43
6.5.1 Análisis . . . 43
6.5.2 Diseño . . . 43
6.5.3 Implementación . . . 49
6.5.4 Pruebas . . . 50
6.6 Iteración 4: Ajuste fino . . . 51
6.6.1 Análisis . . . 52
6.6.2 Diseño . . . 57
6.6.3 Implementación . . . 64
6.6.4 Pruebas . . . 66
6.7 Iteración 5: Interfaz gráfica . . . 69
6.7.1 Análisis . . . 70
6.7.2 Diseño . . . 70
6.7.3 Implementación . . . 71
6.7.4 Pruebas . . . 72

7 Conclusiones **75**

8 Trabajo futuro **77**

Lista de acrónimos **80**

Glosario **81**

Bibliografía **82**

Índice de figuras

5.1 Diagrama de Gantt con planificación inicial 19
5.2 Diagrama de Gantt con planificación final 23

6.1 Arquitectura general del sistema 26
6.2 Casos de uso para iteración 1 28
6.3 Diseño módulos de carga 30
6.4 Diagrama de secuencia para carga de recursos 32
6.5 Casos de uso para iteración 2 37
6.6 Diseño módulo de inferencia 38
6.7 Diagrama de secuencia para inferencia de modelo 40
6.8 Casos de uso para iteración 3 45
6.9 Diseño módulo de procesamiento 46
6.10 Diagrama de secuencia para procesamiento 48
6.11 Casos de uso para iteración 4 52
6.12 Diseño módulo de ajuste fino 58
6.13 Diagrama de secuencia para ajuste fino 61
6.14 Diagrama de secuencia para reanudación de entrenamiento 63
6.15 Diseño módulo de interfaz gráfica 70

Índice de tablas

4.1 Comparativa entre herramientas para ajuste fino y uso de modelos generativos. 13
4.2 Análisis DAFO del sistema desarrollado. 15

5.1 Salarios medios anuales y por hora en Galicia 20
5.2 Estimación de costes de personal. 20
5.3 Coste total estimado del proyecto 22
5.4 Estimación de costes de personal. 24

6.1 Caso de uso CU-01: Cargar recurso 28
6.2 Caso de uso CU-02: Cargar modelo 29
6.3 Caso de uso CU-03: Cargar dataset 29
6.4 Caso de uso CU-04: Desinstalar recurso del catálogo 29
6.5 Caso de uso CU-05: Eliminar recurso del catálogo 29
6.6 Caso de uso CU-06: Limpiar catálogo de recursos 31
6.7 Resultados de test de la primera iteración. 35
6.8 Caso de uso CU-07: Realizar inferencia 37
6.9 Resultados de test de la segunda iteración. 42
6.10 Caso de uso CU-07 con procesamiento: Realizar inferencia 44
6.11 Caso de uso CU-08: Visualizar procesamiento 45
6.12 Resultados de test de la tercera iteración. 51
6.13 Caso de uso CU-07 con adaptadores: Realizar inferencia 53
6.14 Caso de uso CU-09: Realizar ajuste fino 54
6.15 Caso de uso CU-10: Reanudar ajuste fino 55
6.16 Caso de uso CU-11: Ejecutar entrenamiento 56
6.17 Caso de uso CU-12: Eliminar entrenamiento 57
6.18 Caso de uso CU-13: Limpiar catálogo de entrenamientos 57
6.19 Resultados de test de la cuarta iteración. 67
6.20 Resultados de test de la quinta iteración. 72

Capítulo 1

Introducción

En los últimos años la Inteligencia Artificial (IA) [1] ha experimentado un crecimiento exponencial, especialmente en el ámbito de los modelos generativos [2], con un gasto mundial estimado de 644.000 millones de dólares en 2025, lo que supone un aumento del 76,4% con respecto a 2024 [3]. Además, el uso de estos modelos se ha extendido en el día a día. ChatGPT [4], el asistente conversacional de OpenAI [5] y uno de los más populares mundialmente, superó los 900 millones de usuarios activos semanales en febrero de 2026 [6], lo que refleja el gran impacto y presencia que tiene la inteligencia artificial en la vida cotidiana.

Existen diferentes modelos generativos que permiten generar texto, imágenes, audios y/o vídeos mediante distintas modalidades de entrada. Entre estos modelos, destacan los conocidos como *Large Language Model* (LLM) [7] para generación de texto y los modelos de difusión [8] para generación de imágenes. Realizar el entrenamiento de un modelo desde cero presenta diversas limitaciones como su elevado coste computacional y la necesidad de disponer de un *dataset* de gran tamaño, ya que el rendimiento del modelo depende del tamaño del mismo, la cantidad de datos y recursos computacionales empleados en el entrenamiento, lo que dificulta su desarrollo desde cero y fomenta el uso de técnicas como el ajuste fino (también conocido como *fine-tuning*) [9].

Con el ajuste fino se consigue adaptar un modelo preentrenado genérico a una tarea o dominio específico con un conjunto de datos más reducido, disminuyendo así el coste computacional necesario. Sin embargo, el ajuste fino completo sigue siendo un proceso computacionalmente costoso en modelos de gran tamaño, por lo que surgen técnicas como *Low-Rank Adaptation* (LoRA), que introducen un número reducido de parámetros sin modificar los pesos originales del modelo, haciendo el proceso más eficiente [10].

Para realizar estas tareas, una gran cantidad de modelos y *datasets* están alojados en Hug-

ging Face [11]. Esta plataforma actúa como repositorio de modelos preentrenados y conjuntos de datos, así como de librerías que facilitan su utilización.

Actualmente existen librerías como Transformers [12], Diffusers [13] y PEFT [14] de Hugging Face para el uso y entrenamiento de modelos generativos que requieren programación explícita por parte del programador. Además, existen herramientas con interfaz gráfica como, por ejemplo, AUTOMATIC1111 [15] y kohya_ss [16], centradas principalmente en modelos de difusión y en técnicas de ajuste fino concretas, así como LLaMA-Factory [17] y Hugging Face AutoTrain [18], orientadas al ajuste fino de LLM.

Sin embargo, las herramientas anteriormente mencionadas están enfocadas generalmente en un tipo de modelo o técnica en concreto y no constituyen un sistema genérico y extensible que integre de forma unificada todas las fases de ajuste fino como la carga de modelos y *datasets*, la inferencia y el procesado de entradas y salidas, lo que dificulta la capacidad de adaptación a diferentes tipos de modelos, modalidades y técnicas de ajuste fino dentro de una única herramienta con interfaz gráfica.

De esta manera, este proyecto diseña e implementa una aplicación con interfaz gráfica que integra de forma modular las distintas fases del proceso de ajuste fino y uso de modelos generativos de tal forma que permita adaptarse a diferentes tipos de modelos y técnicas. La aplicación permitirá al usuario seleccionar un modelo y un *dataset* mediante Hugging Face o almacenamiento en disco, seleccionar una técnica de ajuste fino con sus respectivos parámetros y realizar el entrenamiento, permitiendo además la inferencia del modelo.

Para validar el correcto funcionamiento del sistema, se implementa un caso de uso basado en modelos de difusión, junto con las técnicas de LoRA y DreamBooth [19], ya que con este caso se permite evaluar la capacidad del sistema de gestionar de forma unificada las distintas fases del ajuste fino, así como su adaptabilidad a diferentes tipos de modelos y técnicas.

1.1 Objetivos

El proyecto tiene como objetivo principal desarrollar un sistema modular y extensible para el uso y ajuste fino de modelos generativos. Este sistema debe ser capaz de adaptarse a distintos tipos de modelos, *datasets* y técnicas e integrar de forma unificada las etapas de carga de modelos y *datasets*, inferencia, procesado y entrenamiento a través de una interfaz gráfica. Para ello, se plantean los siguientes objetivos en concreto:

- Analizar el estado del arte de las herramientas y sistemas actuales que permiten el uso y

el ajuste fino de modelos generativos.

- Diseñar una arquitectura modular que desacople la modalidad del modelo, el tipo de *dataset* y la técnica de ajuste fino.
- Implementar la carga de modelos y *datasets* desde almacenamiento local y desde Hugging Face.
- Integrar de forma unificada las etapas de inferencia, procesado y ajuste fino.
- Validar el sistema mediante la implementación del caso de uso basado en modelos de difusión.
- Desplegar una aplicación que permita la interacción con el sistema a través de una interfaz gráfica.

1.2 Estructura de la memoria

El documento se estructura de la siguiente manera:

- El Capítulo 1, Introducción, presenta la motivación del proyecto, los objetivos planteados y la organización de la memoria.
- El Capítulo 2, Fundamentos teóricos, desarrolla los fundamentos teóricos necesarios para la comprensión del proyecto.
- El Capítulo 3, Materiales y métodos, presenta las herramientas utilizadas para el desarrollo del sistema.
- El Capítulo 4, Estudio de mercado, presenta las alternativas al sistema existentes en el mercado y realiza un análisis DAFO que justifica la necesidad de la solución propuesta.
- El Capítulo 5, Planificación del proyecto, desarrolla la planificación del proyecto, indicando la metodología seguida, la planificación inicial con los recursos humanos, materiales y estimación de costes, y el calendario final.
- El Capítulo 6, Desarrollo del sistema, presenta los requisitos del sistema, la arquitectura general y cada una de las iteraciones, contando para cada una de ellas con una fase de análisis, de diseño, de implementación y de pruebas.
- El Capítulo 7, Conclusiones, presenta las conclusiones obtenidas sobre la realización del proyecto.
- El Capítulo 8, Trabajo futuro, expone las posibles líneas de trabajo futuro y mejoras potenciales del sistema.

Capítulo 2

Fundamentos teóricos

Este capítulo presenta los principales fundamentos teóricos necesarios para la comprensión del proyecto. Primero se describen los modelos generativos, fundamentales al tratarse de la base del proyecto. A continuación, se presenta el ajuste fino, técnica eficiente para el entrenamiento de los modelos generativos y LoRA y DreamBooth como técnicas concretas, siendo estas las técnicas de ajuste fino implementadas inicialmente en el sistema.

2.1 Modelos generativos

Los modelos generativos constituyen una de las principales áreas dentro de la IA [2]. Estos modelos se caracterizan por producir datos nuevos como texto, imagen, audio o vídeo a partir de diferentes patrones obtenidos durante el entrenamiento. A diferencia de los modelos discriminativos, cuyo objetivo es analizar los datos de entrada para realizar tareas como la clasificación o la regresión, los modelos generativos son capaces de generar contenido nuevo que contiene características similares a las presentes en los datos utilizados para su entrenamiento [20]. De esta forma, los modelos generativos han experimentado un gran crecimiento en los últimos años y están presentes en múltiples aplicaciones de uso cotidiano, como asistentes conversacionales, sistemas de generación de imágenes o herramientas de asistencia a la programación.

Además, estos modelos pueden clasificarse desde distintos puntos de vista, de modo que un mismo modelo puede pertenecer a varias categorías de forma simultánea. Por ejemplo, un modelo como Stable Diffusion pertenece a la categoría de modelo de generación de imágenes atendiendo a su modalidad, y a la categoría de modelo de difusión atendiendo a su arquitectura. De esta forma, se distinguen las siguientes clasificaciones:

- Según la **modalidad**:
  - **Modelos de generación de texto**: Generan lenguaje natural o código. Dentro de esta categoría se encuentran los LLM, como GPT, LLaMA, Claude y Gemini.

  - **Modelos de generación de imágenes**: Generan imágenes nuevas generalmente a partir de texto y/o de otra imagen. Se encuentran modelos como Stable Diffusion y DALL-E.
  - **Modelos de generación de audio**: Generan voz, música o sonidos generalmente a partir de una descripción textual o una muestra de audio. Dentro de esta categoría se encuentran modelos como AudioLM y Bark.
  - **Modelos de generación de vídeo**: Generan vídeos a partir de descripciones textuales, imágenes o vídeos de referencia. Se encuentran modelos como Sora y Pika.
  - **Modelos multimodales generativos**: Integran múltiples modalidades de entrada y de salida, permitiendo generar distintos tipos de contenido dentro de un mismo modelo. Se encuentran modelos como GPT-4o y Gemini, capaces de procesar simultáneamente texto, imágenes, audio y otras modalidades.

- Según la **arquitectura**:

  - **Modelos autorregresivos**: Modelos especializados en generar contenido de forma secuencial prediciendo cada elemento a partir de los anteriores [7]. Son ampliamente utilizados en tareas de procesamiento de lenguaje natural y conforman la base de muchos modelos de generación de texto como GPT y LLaMA.
  - **VAEs (*Variational autoencoder*)**: Modelos que aprenden una representación latente comprimida de los datos de entrenamiento y generan muestras nuevas a partir de dicho espacio latente. Son ampliamente utilizados en tareas de generación de imágenes. Arquitecturas como Stable Diffusion incorporan un VAE para manejar representaciones más compactas de las imágenes, reduciendo de esta manera el coste computacional y mejorando la eficiencia del proceso de generación [21].
  - **GANs (*Generative Adversarial Networks*)**: Modelos basados en la competición entre dos redes neuronales, un generador y un discriminador. El primero intenta producir muestras cada vez más realistas y el segundo trata de distinguir entre muestras reales y generadas, mejorando progresivamente la calidad de los resultados [22].
  - **Modelos de difusión**: Generan muestras a partir de un proceso iterativo de eliminación de ruido [8]. Durante el entrenamiento, se añade ruido a las muestras originales y aprenden de forma iterativa a eliminar dicho ruido, permitiendo posteriormente generar nuevas muestras. Este tipo de modelos es ampliamente utilizado para la generación de imágenes, con arquitecturas como Stable Diffusion y Stable Diffusion XL [23].

2.2 Ajuste fino

El entrenamiento de un modelo generativo consiste en ajustar sus parámetros a partir de un conjunto de datos (*dataset*), con el objetivo de aprender patrones presentes en los datos. Por ejemplo, para el caso concreto de modelos para generación de imágenes, se necesitan pares texto-imagen para aprender la relación entre ambas modalidades.

Durante el entrenamiento, el modelo procesa las muestras contenidas en el *dataset* y genera una predicción. Esta se compara con el resultado esperado utilizando una función de pérdida (*loss function*) para medir el error cometido. A partir del error, se actualizan los parámetros entrenables del modelo con el objetivo de mejorar la capacidad del modelo para realizar la tarea.

Los modelos generativos modernos cuentan con millones de parámetros, lo que dificulta realizar un entrenamiento desde cero al requerir una gran cantidad de datos y un elevado coste computacional. Además, el tiempo requerido para completar el proceso también es elevado, dificultando su utilización en entornos limitados [9]. Debido a estas limitaciones, surge el ajuste fino, que permite reutilizar modelos previamente entrenados y adaptarlos a una tarea o dominio específico.

A diferencia del entrenamiento desde cero, el ajuste fino parte de un modelo preentrenado genérico que ya ha aprendido una representación general de los datos, de forma que solamente se necesita especializarlo en una tarea concreta. De esta manera, se reduce la cantidad de datos necesarios, el tiempo de entrenamiento y el coste computacional. Sin embargo, el ajuste fino sigue implicando modificar una gran cantidad de parámetros del modelo y por tanto se sigue precisando de una cantidad considerable de memoria y almacenamiento. Para mitigar estas limitaciones surgen nuevas técnicas de ajuste fino eficiente en parámetros, conocidas como *Parameter-Efficient Fine-Tuning* (PEFT), cuyo objetivo es adaptar el modelo modificando únicamente una pequeña parte de sus parámetros. Entre las técnicas de ajuste fino destacan LoRA [10] y DreamBooth [19], ampliamente utilizadas en modelos generativos.

2.3 LoRA

Low-Rank Adaptation (LoRA) es una técnica *Parameter-Efficient Fine-Tuning* (PEFT) que permite adaptar modelos preentrenados sin modificar los pesos originales. De esta forma, se congelan los parámetros del modelo y se añaden nuevos parámetros entrenables con el objetivo de aprender la nueva tarea. Como consecuencia, se reduce la memoria y el coste computacional, consiguiendo un proceso más rápido y eficiente [10].

Para ello, LoRA en lugar de actualizar la matriz de pesos completa durante el ajuste fino, representa la actualización utilizando dos matrices de bajo rango, obteniendo la matriz de pesos adaptada. Como se muestra en la Ecuación 2.1, los pesos adaptados se obtienen sumando a la matriz original una actualización aprendida durante el ajuste fino:

$$W' = W + BA \tag{2.1}$$

Donde W representa la matriz de pesos original del modelo, W' la matriz de pesos adaptada tras el ajuste fino y A y B las matrices de bajo rango introducidas por LoRA. Durante el entrenamiento únicamente se actualizan los parámetros contenidos en A y B, permaneciendo congelados los pesos originales del modelo. Como consecuencia, el número de parámetros entrenables se reduce considerablemente con respecto al ajuste fino tradicional. Además, una vez terminado el entrenamiento, solamente es necesario almacenar las matrices de bajo rango aprendidas por LoRA, conocidas como adaptadores. Estos adaptadores se pueden cargar posteriormente sobre el modelo original para incorporar lo aprendido durante el entrenamiento sin necesidad de almacenar una copia completa del modelo ajustado.

2.4 DreamBooth

DreamBooth es una técnica de ajuste fino para modelos de difusión que permite enseñar a un modelo sujetos, objetos o estilos específicos a partir de un *dataset* de tamaño reducido. Para ello, se dispone de varias imágenes del concepto a aprender junto con una descripción textual. Las descripciones textuales incorporan un identificador único común que permite al modelo asociar dicho identificador con el concepto aprendido. Durante el entrenamiento, el modelo aprende las características visuales de las imágenes, permitiendo generar nuevas imágenes con el concepto aprendido [19].

Sin embargo, entrenar solamente con imágenes del nuevo concepto presenta el riesgo de que el modelo pierda parte del conocimiento adquirido durante el preentrenamiento. Para evitar este problema, DreamBooth utiliza el mecanismo *prior preservation*, que consiste en incluir ejemplos de la clase general a la que pertenece el concepto aprendido durante el entrenamiento. De esta forma, el modelo puede aprender las características específicas del nuevo concepto sin perder su capacidad para generar ejemplos generales de dicha clase. Por ejemplo, si el concepto a aprender es una mascota en concreto, además de utilizar imágenes de dicha mascota se incluyen imágenes generales de la clase a la que pertenece, como perro o gato [19]. Actualmente, DreamBooth suele combinarse con técnicas PEFT como LoRA para reducir los requisitos computacionales y de almacenamiento del entrenamiento.

Capítulo 3

Materiales y métodos

Este capítulo presenta los entornos de desarrollo utilizados para el proyecto, indicando para cada uno de ellos una descripción y su utilidad en el proyecto. Primero se introduce Python, lenguaje de programación ampliamente utilizado para desarrollo de aplicaciones de IA. A continuación se presenta Hugging Face, plataforma utilizada para la obtención y gestión de modelos y *datasets* y, por último, *Streamlit*, utilizado para el desarrollo de la interfaz gráfica.

3.1 Python

El lenguaje empleado para el desarrollo del sistema es *Python* [24]. Este es un lenguaje de programación interpretado, de alto nivel y ampliamente utilizado en el desarrollo de aplicaciones *software* y en sistemas de inteligencia artificial. La elección de *Python* se debe principalmente a la disponibilidad de librerías especializadas para la carga, inferencia y ajuste fino de modelos generativos, como *PyTorch*, *Transformers*, *Diffusers* y *PEFT* [12, 13, 14, 25].

Para la implementación del sistema se emplean diferentes librerías especializadas en la carga de modelos y *datasets*, la inferencia, el ajuste fino y el desarrollo de la interfaz gráfica. Las principales librerías utilizadas son las siguientes:

- **PyTorch** [25]: Biblioteca de aprendizaje profundo utilizada para la representación tensorial, ejecución de modelos, entrenamiento, inferencia y uso de aceleradores *hardware* como GPU.
- **Diffusers** [13]: Librería especializada en los modelos de difusión. Proporciona funciones para la carga, inferencia y ajuste fino de arquitecturas como Stable Diffusion y Stable Diffusion XL.
- **Transformers** [12]: Librería utilizada para modelos basados en la arquitectura Transformer. Se utiliza para la carga y uso de modelos generativos de texto y componentes

textuales empleados por otros modelos generativos.

- **PEFT** [14]: Librería utilizada para técnicas de ajuste fino eficiente en parámetros. Se utiliza para el uso de adaptadores LoRA.
- **Datasets** [26]: Librería utilizada para la carga y gestión de *datasets*.
- **Accelerate** [27]: Librería utilizada para simplificar la ejecución eficiente de modelos y procesos de entrenamiento sobre distintos dispositivos hardware.
- **Safetensors** [28]: Librería utilizada para el almacenamiento de pesos de modelos y de adaptadores.
- **Pillow** [29]: Librería utilizada para el procesamiento de imágenes.

3.2 Hugging Face

Hugging Face [11] es una plataforma destinada al desarrollo y distribución de modelos de aprendizaje automático que proporciona repositorios de modelos, *datasets* y herramientas para su utilización. Hoy en día es ampliamente utilizada dentro de la comunidad de inteligencia artificial y es una de las principales plataformas de distribución de modelos generativos y de *datasets*.

A través de Hugging Face Hub, los usuarios pueden publicar y descargar modelos preentrenados y conjuntos de datos. Además de actuar como repositorio de recursos para el ámbito de los modelos generativos, también proporciona APIs y librerías que permiten acceder y trabajar con dichos recursos desde *Python*. Entre las librerías más utilizadas se encuentran *transformers*, *diffusers* y *datasets*, mencionadas en la Sección 3.1, que permiten cargar modelos y *datasets*, ejecutar operaciones de inferencia y realizar ajustes finos y entrenamientos.

Con respecto a su utilidad en el proyecto, Hugging Face se emplea como una de las fuentes principales para la obtención de modelos y de *datasets*. Asimismo, sus librerías facilitan la descarga, carga y reutilización de recursos preentrenados, así como la ejecución de procesos de inferencia y ajuste fino.

3.3 Streamlit

Streamlit es un *framework* de código abierto para el desarrollo de aplicaciones web basado en Python [30]. Permite desarrollar interfaces gráficas de forma sencilla mediante componentes predefinidos, de forma que no es necesario utilizar las tecnologías web tradicionales como HTML, CSS o JavaScript. Es ampliamente utilizado en el ámbito de la inteligencia artificial

debido a su integración con el ecosistema Python y a la rapidez con la que permite desarrollar aplicaciones interactivas.

Capítulo 4

Estudio de mercado

Este capítulo realiza un análisis de las herramientas actuales que permiten utilizar y realizar ajuste fino de modelos generativos, comparando asimismo la solución propuesta con dichas herramientas con el objetivo de identificar sus principales diferencias. Además, se realiza un análisis DAFO para evaluar la situación interna y externa del proyecto.

4.1 Herramientas existentes

En esta sección se presentan las principales herramientas actuales utilizadas en el ámbito de los modelos generativos, distinguiendo entre las especializadas en la inferencia y el ajuste fino.

4.1.1 Herramientas para inferencia

Con respecto a la inferencia de modelos generativos, existen varias herramientas que permiten utilizar modelos preentrenados mediante interfaces gráficas sin la necesidad de tener que realizar programación explícita. Estas herramientas permiten gestionar y cargar modelos, además de realizar inferencias con diferentes entradas, simplificando el proceso mediante interfaces gráficas.

- **AUTOMATIC1111** [15]: Interfaz web desarrollada para Stable Diffusion que proporciona funcionalidades para la generación de imágenes a partir de texto y gestión de modelos. Dispone además de diferentes extensiones que amplían sus funcionalidades, incorporando soporte para adaptadores como LoRA.
- **ComfyUI** [31]: Aplicación para modelos de difusión que permite crear flujos de inferencia mediante una interfaz gráfica basada en nodos. Esta aplicación proporciona una gran flexibilidad para combinar distintos modelos, adaptadores y operaciones de procesamiento dentro de un mismo flujo de trabajo.

- **LM Studio** [32]: Aplicación de escritorio que permite descargar, gestionar y ejecutar localmente modelos de lenguaje mediante una interfaz gráfica.
- **Open WebUI** [33]: Interfaz web similar a la de asistentes conversacionales. Permite interactuar con diferentes modelos de lenguaje mediante una interfaz unificada, facilitando su uso por parte de usuarios no técnicos.
- **Jan** [34]: Aplicación de escritorio que permite la descarga y ejecución local de modelos de lenguaje mediante una interfaz sencilla orientada a la privacidad.
- **text-generation-webui** [35]: Herramienta para modelos de lenguaje que destaca por su capacidad de cargar modelos utilizando motores subyacentes y por la posibilidad de ampliar su capacidad mediante extensiones.
- **Ollama** [36]: Herramienta por línea de comandos que simplifica la descarga y ejecución de modelos de lenguaje localmente. Aunque a diferencia del resto de herramientas mencionadas anteriormente no presenta interfaz gráfica, actúa como motor de inferencia fundamental para plataformas como Open WebUI.

4.1.2 Herramientas para ajuste fino

Además de las herramientas enfocadas en la ejecución de inferencias, existen aplicaciones para el ajuste fino de modelos preentrenados. Estas herramientas permiten adaptar modelos genéricos a tareas o dominios específicos mediante diferentes técnicas de entrenamiento, proporcionando interfaces que simplifican la configuración de hiperparámetros y procesos de entrenamiento.

- **kohya_ss** [16]: Una de las herramientas más utilizadas para el ajuste fino de modelos de difusión. Proporciona soporte para diferentes técnicas de ajuste fino como LoRA y DreamBooth, facilitando su uso mediante una interfaz gráfica intuitiva que permite configurar los hiperparámetros correspondientes de manera sencilla. Destaca su uso para modelos basados en Stable Diffusion.
- **Hugging Face AutoTrain** [18]: Plataforma desarrollada por Hugging Face que permite entrenar modelos de forma sencilla, proporcionando soporte para diferentes tareas y arquitecturas de aprendizaje automático, incluyendo modelos de lenguaje, y simplificando el proceso de configuración y ejecución.
- **LLaMA-Factory** [17]: Herramienta utilizada en el ámbito de modelos de lenguaje. Proporciona soporte para los principales modelos de lenguaje como LLaMA, Qwen y Phi junto con técnicas de ajuste fino como LoRA y QLoRA, facilitando la configuración y ejecución de entrenamientos mediante una interfaz gráfica.

- **OneTrainer** [37]: Aplicación de escritorio que unifica el entrenamiento de diferentes arquitecturas de generación de imágenes en una interfaz gráfica, destacando por el control proporcionado sobre el proceso de entrenamiento.
- **Axolotl** [38]: Herramienta por línea de comandos que automatiza el entrenamiento de LLM, centralizando la configuración de hiperparámetros, modelos y *datasets* en un único archivo en formato YAML.

4.2 Comparativa

En esta sección, se muestra una tabla comparativa entre las soluciones existentes y el sistema, abordando las principales características como la gestión de modelos y *datasets*, el soporte para el ajuste fino e inferencia de modelos generativos, la presencia de operaciones de procesamiento, interfaz gráfica y dependencia de arquitecturas.

Tabla 4.1: Comparativa entre herramientas para ajuste fino y uso de modelos generativos.

Herramienta	**Gestión modelos**	**Gestión datasets**	**Inferencia**	**Procesamiento**	**Ajuste fino**	**Interfaz gráfica**	**Dependencia de arquitectura o modalidad**
AUTOMATIC1111	Sí	No	Sí	Parcial	Parcial	Sí	Modelos de difusión (Stable Diffusion)
ComfyUI	Sí	No	Sí	Sí	Parcial	Sí	Principalmente modelos de difusión
LM Studio	Sí	No	Sí	No	No	Sí	Modelos de lenguaje (LLM)
Open WebUI	Sí	No	Sí	Parcial	No	Sí	Modelos de lenguaje (LLM)
kohya_ss	Sí	Sí	No	No	Sí	Sí	Modelos de difusión (Stable Diffusion)
AutoTrain	Sí	Sí	No	No	Sí	Sí	Ecosistema Hugging Face
LLaMA-Factory	Sí	Sí	Sí	No	Sí	Sí	Modelos de lenguaje y visión-lenguaje
Jan	Sí	No	Sí	No	No	Sí	Modelos de lenguaje (LLM)
text-generation-webui	Sí	No	Sí	No	Parcial	Sí	Modelos de lenguaje (LLM)

(continúa en la página siguiente)

Tabla 4.1 – *(viene de la página anterior)*

Herramienta	**Gestión modelos**	**Gestión datasets**	**Inferencia**	**Procesamiento**	**Ajuste fino**	**Interfaz gráfica**	**Dependencia de arquitectura o modalidad**
Ollama	Sí	No	Sí	No	No	No	Modelos de lenguaje (LLM)
OneTrainer	Sí	Sí	No	Parcial	Sí	Sí	Modelos de difusión (Stable Diffusion)
Axolotl	Sí	Sí	No	Parcial	Sí	No	Modelos de lenguaje (LLM)
Sistema propuesto	**Sí**	**Sí**	**Sí**	**Sí**	**Sí**	**Sí**	**Arquitectura extensible y modular**

Como se puede observar en la Tabla 4.1, las herramientas existentes suelen estar orientadas a tipos de modelos en concreto. Por una parte, AUTOMATIC1111, ComfyUI, kohya_ss y OneTrainer están enfocadas principalmente a modelos de difusión, mientras que LM Studio, Open WebUI, LLaMA-Factory, Ollama, Jan, text-generation-webui y Axolotl se centran en modelos de lenguaje. Esto presenta ventajas como una mejor adaptación a la arquitectura en concreto, pudiendo ofrecer funcionalidades muy específicas de la arquitectura, pero limita el poder ofrecer más variedad de modelos y técnicas.

Por otra parte, muchas de estas herramientas están enfocadas en una tarea en concreto, de forma que algunas cuentan con la inferencia como funcionalidad principal y otras el ajuste fino. Para solucionar esto, presentan extensiones para mejorar e incorporar funcionalidades, pero es necesario instalarlas y no son proporcionadas de manera nativa. Además, algunas herramientas no cuentan con una interfaz gráfica que simplifique el proceso de ajuste de parámetros de inferencia o hiperparámetros de entrenamiento, por lo que requieren una programación explícita o el uso de archivos de configuración.

De esta manera, el sistema desarrollado integra de forma unificada la gestión de modelos y *datasets*, inferencia, procesamiento y ajuste fino, presentando una arquitectura modular y extensible que facilita la incorporación de nuevas arquitecturas, técnicas de ajuste fino y operaciones de procesamiento sin la necesidad de modificar el núcleo del sistema, mediante una interfaz gráfica.

4.3 Análisis DAFO

En esta sección se presenta un análisis DAFO (Debilidades, Amenazas, Fortalezas y Oportunidades) centrado en un sistema para el uso y ajuste fino de modelos generativos. Este análisis permite identificar los factores clave que inciden en la viabilidad técnica, social y estratégica del proyecto.

Fortalezas	**Debilidades**
• Arquitectura modular y extensible. • Integración de inferencia, procesamiento y ajuste fino. • Gestión de modelos y *datasets*. • Interfaz gráfica sencilla y funcional.	• Menor especialización que herramientas orientadas a una arquitectura concreta.
Oportunidades	**Amenazas**
• Crecimiento del uso de modelos generativos. • Aparición de nuevas arquitecturas y modalidades de modelos generativos. • Necesidad de herramientas que unifiquen inferencia y ajuste fino de modelos.	• Evolución rápida de arquitecturas y técnicas. • Aparición de nuevas herramientas especializadas.

Tabla 4.2: Análisis DAFO del sistema desarrollado.

Como se refleja en la Tabla 4.2, el análisis muestra un entorno favorable para el desarrollo de un sistema modular y extensible para el ajuste fino y uso de modelos generativos, caracterizado por el crecimiento exponencial que presenta la inteligencia artificial y, en particular, el ámbito de los modelos generativos. Asimismo, la gran variedad de modalidades y técnicas de ajuste fino existentes, junto con la continua aparición de nuevos modelos y técnicas de entrenamiento, justifican la necesidad de una herramienta capaz de adaptarse a las diferencias entre arquitecturas y técnicas, así como a la constante evolución que presenta el campo.

4.4 Conclusión

El análisis realizado expone las limitaciones que presentan las herramientas actuales, especializadas por lo general en arquitecturas, técnicas o tareas concretas. El campo de los modelos generativos está en constante evolución y cada vez surgen más arquitecturas y técnicas nuevas, lo que sugiere la necesidad de buscar una herramienta capaz de adaptarse a la continua evolución y que unifique las tareas de inferencia y ajuste fino abstrayendo las particularidades de cada modelo y técnica.

En este contexto, el sistema desarrollado propone una arquitectura modular y extensible que integra de forma unificada la gestión de recursos (modelos y *datasets*), la inferencia, el procesamiento y el ajuste fino desde una única aplicación con interfaz gráfica, facilitando la incorporación de nuevas arquitecturas y técnicas de ajuste fino que surjan en el futuro sin la necesidad de modificar el núcleo del sistema.

Capítulo 5

Planificación del proyecto

Este capítulo describe la planificación del proyecto, introduciendo primero la metodología a seguir durante el desarrollo del mismo, la planificación inicial con los recursos humanos, materiales y costes y, por último, el calendario final.

5.1 Metodología

Para la realización del proyecto se sigue una **metodología iterativa incremental** [39]. Esta metodología consiste en organizar el desarrollo del sistema en ciclos sucesivos, denominados iteraciones, en lugar de desarrollar el sistema completo en una única fase. En cada iteración se realizan actividades de análisis, diseño, implementación y pruebas sobre una parte concreta del sistema. Como resultado, cada iteración genera un incremento funcional o prototipo parcial que amplía la versión anterior y puede ser validado antes de continuar con el desarrollo.

De esta manera, la metodología iterativa incremental se adapta adecuadamente a las características esenciales del proyecto, al tratarse de un sistema genérico, extensible y modular. La división en iteraciones permite la sucesiva incorporación de funcionalidades, estableciendo primero las bases del sistema y añadiendo posteriormente nuevos componentes apoyándose en los anteriores.

Como consecuencia, se distinguen cinco iteraciones principales correspondientes a la gestión de modelos y *datasets*, la inferencia, el procesamiento, el ajuste fino y la interfaz gráfica. Estas iteraciones permiten organizar el desarrollo del sistema de forma progresiva, incorporando nuevas funcionalidades sobre las iteraciones anteriores ya validadas.

5.2 Planificación inicial

La planificación inicial del proyecto se realiza teniendo en cuenta la complejidad de cada una de las funcionalidades a desarrollar, así como la experiencia previa en uso y ajuste fino de modelos generativos. Estas estimaciones se han empleado para establecer tanto el cronograma del proyecto como el coste asociado a cada actividad.

Recursos humanos

Se identifican dos perfiles esenciales:

- **Desarrollador de software (DS)**: Responsable del análisis de requisitos, diseño de la arquitectura del sistema, implementación de los distintos módulos, desarrollo de la interfaz gráfica, integración de los componentes y validación del sistema.
- **Ingeniero de inteligencia artificial (IA)**: Encargado de la selección e integración de modelos generativos, gestión de modelos y *datasets*, implementación del módulo de inferencia, ajuste fino y evaluación de los resultados.

El proyecto se divide en cinco iteraciones, en las que cada una de ellas cuenta con una fase de análisis, de diseño, de implementación y de pruebas. La gestión temporal de los recursos humanos involucrados en el desarrollo del sistema se recoge en el diagrama de Gantt de la Figura 5.1:

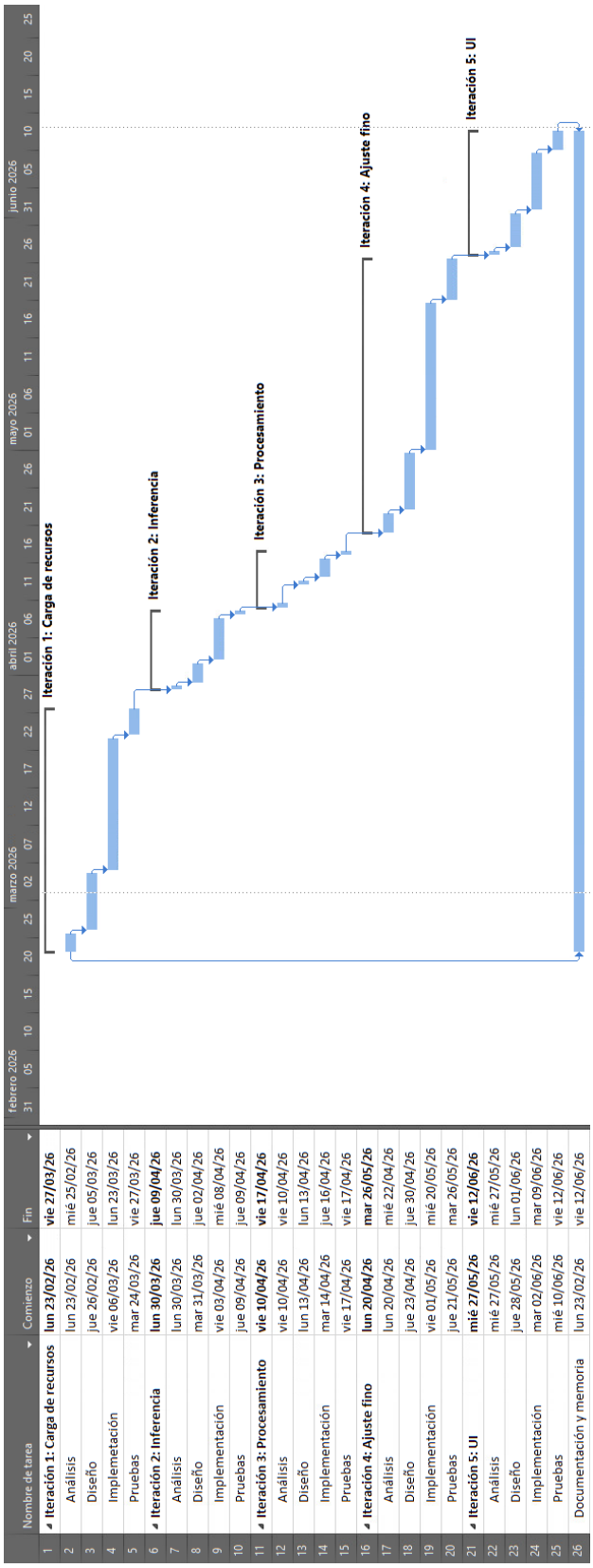


Figura 5.1: Diagrama de Gantt con planificación inicial

Recursos materiales

Para el desarrollo del proyecto se dispone de los siguientes recursos materiales:

- **Ordenador portátil personal**: Utilizado para el desarrollo, pruebas y ejecución de las diferentes operaciones del sistema. El dispositivo cuenta con un procesador Intel Core i7-12700H de 12.ª generación, 16 GB de memoria RAM, almacenamiento SSD de aproximadamente 512 GB y Windows 11 como sistema operativo.
- **Unidad de procesamiento gráfico (GPU)**: Utilizada para acelerar el proceso de las operaciones de inferencia y ajuste fino de los modelos generativos. En concreto, se ha empleado una GPU NVIDIA GeForce RTX 3050 integrada en el equipo de desarrollo.
- **Capacidad de almacenamiento local**: Destinada al almacenamiento de los modelos generativos y *datasets* utilizados para las pruebas, y para los adaptadores y resultados generados durante los entrenamientos.
- **Entorno de software**: Compuesto por el lenguaje de programación Python, las librerías PyTorch, Transformers, Diffusers y Datasets, además del framework Streamlit para el desarrollo de la interfaz gráfica.

Estimación de costes

Los costes del personal se calculan basándose en los esfuerzos estimados en el cronograma del proyecto, así como en las tasas salariales medias en Galicia para los perfiles implicados, extraídas de Glassdoor [40]. Los datos se muestran en la Tabla 5.1.

Perfil	Salario anual (€)	Coste por hora (€)
Desarrollador de software	35.000	16,83
Ingeniero de IA	40.000	19,23

Tabla 5.1: Salarios medios anuales y por hora en Galicia

Teniendo en cuenta estos valores, se calcula el coste por tarea, mostrado en la Tabla 5.2. El esfuerzo total corresponde con **352 horas**, con un coste del personal global de **6.365,76 €**.

Tabla 5.2: Estimación de costes de personal.

Tarea	Perfil	Esfuerzo (h)	Coste (€)
Iteración 1: Gestión de modelos y datasets			
Análisis	DS	12	201,96
Diseño	DS	24	403,92
Implementación	DS	48	807,84
Pruebas	DS	16	269,28

. . . . . *(continúa en la página siguiente)* . . . . . .

Tabla 5.2 – *(viene de la página anterior)*

Tarea	Perfil	Esfuerzo (h)	Coste (€)
Iteración 2: Inferencia			
Análisis	IA	4	76,92
Diseño	IA	12	230,76
Implementación	IA	16	307,68
Pruebas	IA	4	76,92
Iteración 3: Procesamiento			
Análisis	IA	4	76,92
Diseño	IA	4	76,92
Implementación	IA	12	230,76
Pruebas	IA	4	76,92
Iteración 4: Ajuste fino			
Análisis	IA	12	230,76
Diseño	IA	24	461,52
Implementación	IA	56	1076,88
Pruebas	IA	16	307,68
Iteración 5: Interfaz gráfica			
Análisis	DS	4	67,32
Diseño	DS	12	201,96
Implementación	DS	24	403,92
Pruebas	DS	12	201,96
Documentación y memoria			
Redacción de memoria	DS e IA	32	576,96
Total		**352**	**6.365,76**

Además de los costes del personal, se consideran los recursos materiales empleados durante el desarrollo. El equipo utilizado corresponde con un portátil personal con un coste aproximado de 1100 €, considerando una vida útil de 5 años. Dado que el desarrollo del proyecto tiene una duración aproximada de 4 meses, el coste imputable al proyecto corresponde aproximadamente con 70 €.

Por otra parte, el software empleado, incluyendo tanto el lenguaje de programación Python como las librerías utilizadas, detalladas en la Sección 3.1, son de código abierto y libres, lo que supone un coste de 0 €. Asimismo, los modelos generativos y *datasets* utilizados proceden de Hugging Face, por lo que no se aumentan los costes. De esta manera, el coste es de **6.435,76 €**, como se muestra en la Tabla 5.3.

Concepto	Coste (€)
Costes de personal	6.365,76
Costes materiales	70,00
Total	**6.435,76**

Tabla 5.3: Coste total estimado del proyecto

5.3 Calendario final

Con respecto a la estimación inicial, se produjeron desviaciones significativas que derivaron en el desplazamiento de la fecha de finalización del proyecto. En la Figura 5.2 se muestra el diagrama de Gantt actualizado, con los tiempos reales de desarrollo del sistema.

Gran parte de las desviaciones fueron causadas por coincidir temporalmente con la dedicación de los recursos humanos en otros proyectos, así como la subestimación de la carga de trabajo en la planificación inicial sobre las tareas relacionadas con los módulos de carga y ajuste fino. Tras las desviaciones sufridas en la estimación inicial, el esfuerzo final corresponde con **380 horas**, con un coste del personal final de **6.875,40 €**, lo que supone un aumento del 8% con respecto al coste estimado inicialmente. El detalle de los costes se muestra en la Tabla 5.4.

Además, considerando los costes materiales empleados en el desarrollo del proyecto, el coste final asciende a **6.945,40 €**.

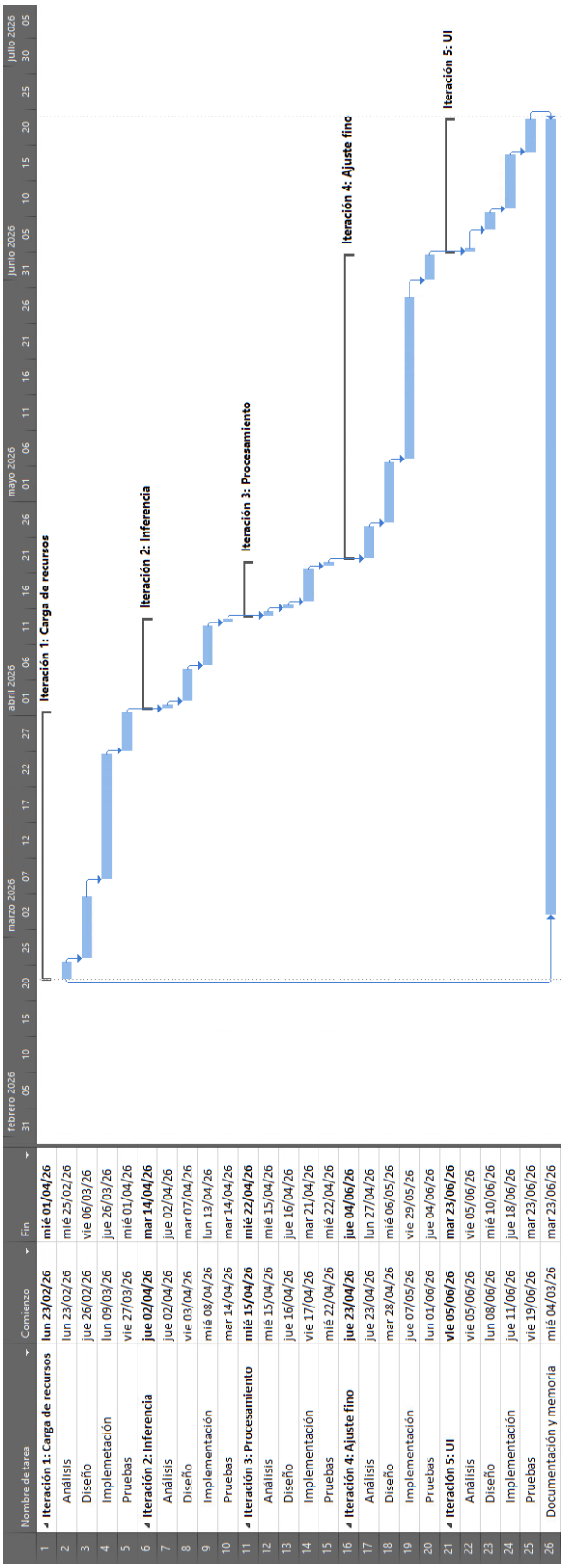


Figura 5.2: Diagrama de Gantt con planificación final

Tabla 5.4: Estimación de costes de personal.

Tarea	Perfil	Esfuerzo (h)	Coste (€)
Iteración 1: Gestión de modelos y datasets			
Análisis	DS	12	201,96
Diseño	DS	28	471,24
Implementación	DS	56	942,48
Pruebas	DS	16	269,28
Iteración 2: Inferencia			
Análisis	IA	4	76,92
Diseño	IA	12	230,76
Implementación	IA	16	307,68
Pruebas	IA	4	76,92
Iteración 3: Procesamiento			
Análisis	IA	4	76,92
Diseño	IA	4	76,92
Implementación	IA	12	230,76
Pruebas	IA	4	76,92
Iteración 4: Ajuste fino			
Análisis	IA	12	230,76
Diseño	IA	28	538,44
Implementación	IA	68	1307,64
Pruebas	IA	16	307,68
Iteración 5: Interfaz gráfica			
Análisis	DS	4	67,32
Diseño	DS	12	201,96
Implementación	DS	24	403,92
Pruebas	DS	12	201,96
Documentación y memoria			
Redacción de memoria	DS e IA	32	576,96
Total		**380**	**6.875,40**

Capítulo 6

Desarrollo del sistema

Este capítulo describe el desarrollo del sistema, comenzando por los requisitos y la arquitectura general, y continuando con la explicación de cada una de las diferentes iteraciones realizadas durante el desarrollo del sistema.

6.1 Requisitos

En esta sección se describen los requisitos del sistema, diferenciando entre los funcionales y los no funcionales. Por una parte, en los requisitos funcionales se definen las capacidades y operaciones que debe proporcionar el sistema, como la carga de recursos, la realización de inferencias, los procesamientos y la aplicación de técnicas de ajuste fino. Por otra parte, los requisitos no funcionales establecen las propiedades y restricciones que debe cumplir el sistema, como la modularidad, extensibilidad y eficiencia.

6.1.1 Requisitos funcionales

- *RF1*: El sistema debe permitir la carga de modelos y *datasets* desde diferentes fuentes.
- *RF2*: El sistema debe permitir la gestión de un catálogo de recursos previamente utilizados.
- *RF3*: El sistema debe permitir realizar inferencia sobre modelos generativos.
- *RF4*: El sistema debe permitir aplicar operaciones de procesamiento sobre las entradas y salidas del modelo.
- *RF5*: El sistema debe permitir aplicar técnicas de ajuste fino sobre modelos.
- *RF6*: El sistema debe permitir gestionar entrenamientos de ajuste fino, incluyendo su registro, reanudación y cancelación.
- *RF7*: El sistema debe permitir cargar y descargar adaptadores compatibles con un modelo para su uso en inferencia.

6.1.2 Requisitos no funcionales

- *RNF1*: El sistema debe ser modular, organizando sus componentes en módulos independientes y desacoplados.
- *RNF2*: El sistema debe ser extensible, permitiendo la incorporación de nuevas fuentes de recursos, tipos de modelos, datasets y técnicas.
- *RNF3*: El sistema debe proporcionar una interfaz gráfica sencilla e intuitiva para el usuario.
- *RNF4*: El sistema debe realizar las operaciones de inferencia y entrenamiento de forma eficiente, optimizando los recursos computacionales y reduciendo los tiempos de ejecución.

6.2 Arquitectura general

Para cumplir con el requisito RNF1 definido en la Sección 6.1.2, se ha diseñado una arquitectura modular basada en componentes independientes en la que cada uno de ellos se encarga de una tarea específica. De esta forma, el sistema cuenta con los siguientes módulos: interfaz gráfica, carga de modelos, carga de datasets, ajuste fino, inferencia y procesado, además del orquestador encargado de la comunicación entre los diferentes módulos. En la Figura 6.1 se muestra la arquitectura general del sistema.

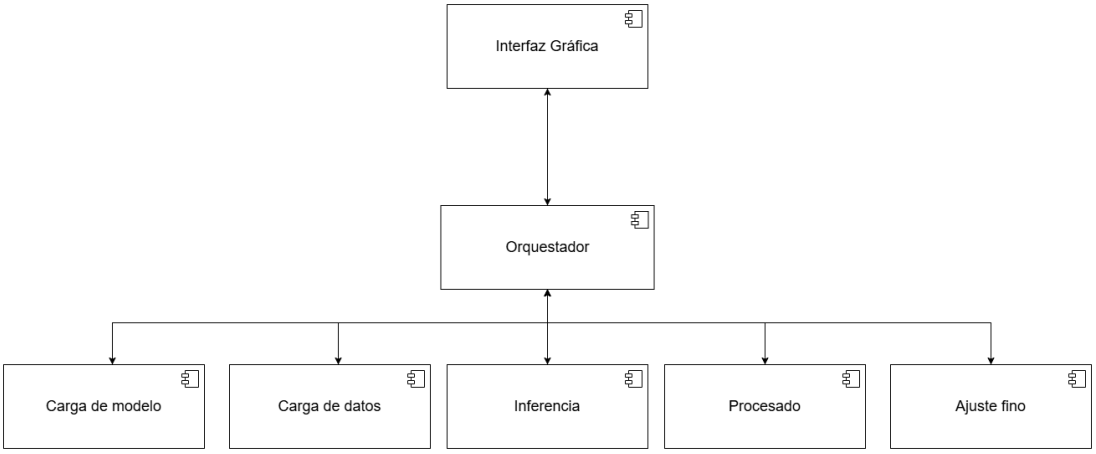


Figura 6.1: Arquitectura general del sistema

La interfaz gráfica se encarga de la interacción del usuario, permitiendo hacer uso de las diferentes funcionalidades del sistema. El orquestador constituye el elemento central encargado de manejar la comunicación entre los diferentes módulos. Los módulos de carga de modelos y de *datasets* manejan la obtención y gestión de los recursos. El módulo de ajuste fino permite realizar el entrenamiento mediante la técnica seleccionada y el módulo de inferencia probar el modelo entrenado o un modelo seleccionado. Por último, el módulo de procesado se encarga de aplicar operaciones de procesamiento a las entradas y salidas del sistema en la inferencia.

6.3 Iteración 1: Carga de modelos y *datasets*

El objetivo de esta iteración es desarrollar los módulos de carga de modelos y de *datasets* definidos en la Sección 6.2, cumpliendo con los requisitos funcionales RF1 y RF2. Estos módulos se encargan de la resolución, carga y gestión de recursos desde diferentes fuentes, así como la gestión de un catálogo de los recursos utilizados en el sistema, diferenciando entre recursos disponibles y no disponibles en local. De esta forma, el sistema puede tanto incorporar nuevos recursos como reutilizar recursos previamente descargados o reinstalar aquellos eliminados localmente. Esta iteración constituye la base necesaria para el resto de funcionalidades del sistema, ya que tanto el módulo de inferencia como el de ajuste fino dependen de la correcta obtención y preparación de los recursos utilizados. Los módulos se desarrollan como componentes diferenciados dentro de la arquitectura modular del sistema, siguiendo el requisito no funcional RNF1. Además, para mantener la extensibilidad del sistema definida en el requisito RNF2, se desacopla la resolución y preparación de los recursos de su posterior carga y utilización en el sistema. Esto facilita la futura incorporación de nuevas fuentes para la obtención de recursos, así como de nuevos tipos de modelos y *datasets*, sin necesidad de modificar el núcleo del sistema ni el resto de módulos.

6.3.1 Análisis

En el diagrama de casos de uso se identifican diferentes elementos. Por una parte, se encuentra el actor **Usuario** que hace referencia al usuario que haga uso del sistema. Este podrá cargar un modelo y dataset nuevos, seleccionar uno previamente utilizado del catálogo y también desinstalar un recurso para liberar almacenamiento en disco, eliminar un recurso del catálogo para eliminar tanto la referencia como el almacenamiento en disco, si corresponde, y por último limpiar el catálogo. Estos elementos, tanto el actor como los casos de uso, se muestran en la Figura 6.2.

En las Tablas 6.1 a 6.6 se muestran los casos de uso desglosados, en donde se muestra para cada uno de ellos una descripción, las precondiciones, la secuencia normal y las postcondiciones.

6.3.2 Diseño

Para los módulos de carga se diseñó una arquitectura que separe la resolución y preparación del recurso de su posterior carga en el sistema. El diseño se muestra en la Figura 6.3.

Como se muestra en la Figura 6.3, el módulo cuenta con los siguientes componentes principales:

1. **Services**: Los servicios exponen las diferentes operaciones que podrá realizar el usuario descritas en la Sección 6.3.1 , distinguiendo por una parte el servicio para modelos y el servicio para *datasets*. Estos se encargarán de coordinar las dos fases de resolución y preparación y la posterior carga.
2. **ResourceSourceResolver**: Para la resolución del origen de los recursos, se sigue el patrón **Estrategia**, mediante el cual cada fuente hereda de la interfaz `ResourceSource` e implementa los métodos de `can_resolve` y `resolve` que usará `ResourceSourceResolver` para resolver el origen del recurso con el que corresponda. Ante la naturaleza diferente entre modelos

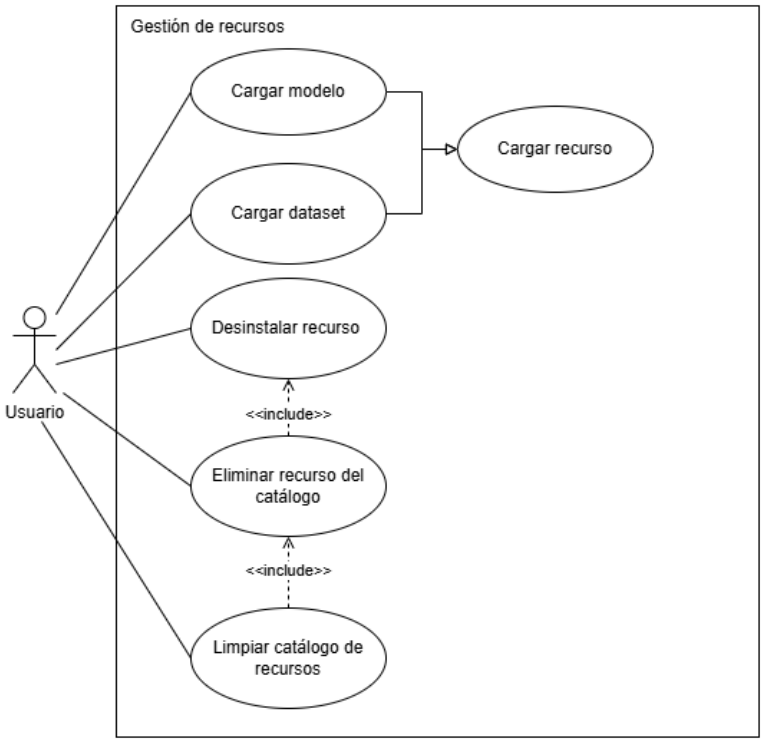


Figura 6.2: Casos de uso para iteración 1

CU-01: Cargar recurso	
Descripción	Permite cargar un recurso en el sistema, indicando un identificador del recurso.
Precondiciones	El sistema debe estar operativo y debe existir acceso al recurso especificado por el usuario.
Secuencia normal	1. El usuario indica el recurso que desea cargar. 2. El sistema busca el recurso en el catálogo. 3. En caso de no estar disponible o estar desinstalado, lo obtiene y prepara a partir del identificador. 4. El sistema carga el recurso y actualiza el catálogo.
Postcondiciones	El recurso está cargado en el sistema y registrado en el catálogo.

Tabla 6.1: Caso de uso CU-01: Cargar recurso

CU-02: Cargar modelo	
Descripción	Especialización del caso de uso CU-01: Cargar recurso para modelos.

Tabla 6.2: Caso de uso CU-02: Cargar modelo

CU-03: Cargar dataset	
Descripción	Especialización del caso de uso CU-01: Cargar recurso para datasets.

Tabla 6.3: Caso de uso CU-03: Cargar dataset

CU-04: Desinstalar recurso del catálogo	
Descripción	Permite desinstalar un recurso registrado en el catálogo.
Precondiciones	El recurso debe estar registrado en el sistema, disponible en local y gestionado por el sistema.
Secuencia normal	1. El usuario selecciona el recurso gestionado del catálogo que desea desinstalar. 2. El recurso se elimina de local. 3. El sistema actualiza el estado del recurso.
Postcondiciones	El recurso queda registrado en el catálogo como no instalado y deja de estar disponible en local.

Tabla 6.4: Caso de uso CU-04: Desinstalar recurso del catálogo

CU-05: Eliminar recurso del catálogo	
Descripción	Permite eliminar completamente un recurso registrado en el catálogo.
Precondiciones	El recurso debe estar registrado en el sistema.
Secuencia normal	1. El usuario selecciona el recurso del catálogo que desea eliminar. 2. En caso de estar instalado y gestionado por el sistema, este desinstala el recurso de local. 3. El sistema elimina la referencia del recurso del catálogo.
Postcondiciones	El recurso deja de estar registrado en el catálogo. Si estaba gestionado por el sistema, deja de estar disponible también en local.

Tabla 6.5: Caso de uso CU-05: Eliminar recurso del catálogo

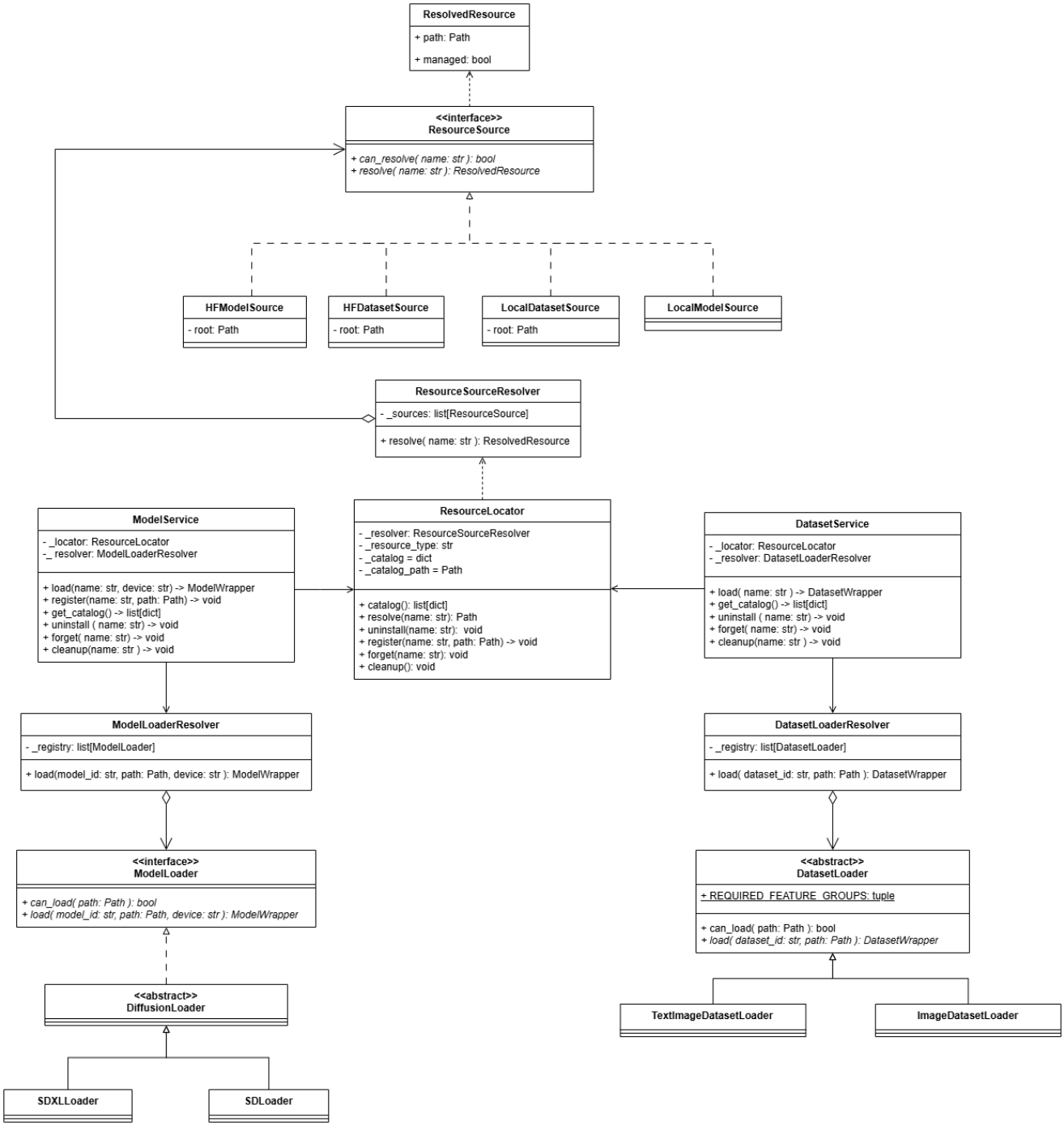


Figura 6.3: Diseño módulos de carga

CU-06: Limpiar catálogo de recursos	
Descripción	Permite eliminar todos los recursos registrados en el catálogo.
Precondiciones	El sistema debe estar operativo.
Secuencia normal	1. El usuario solicita limpiar el catálogo. 2. El sistema desinstala los recursos gestionados por el sistema que estén disponibles en local. 3. El sistema elimina todas las referencias del catálogo.
Postcondiciones	El catálogo queda vacío. Los recursos gestionados por el sistema son eliminados de local, mientras que los recursos externos no gestionados permanecen en su ubicación original.

Tabla 6.6: Caso de uso CU-06: Limpiar catálogo de recursos

y *datasets*, se definen diferentes fuentes para cada tipo de recursos de forma que cada uno pueda adaptar su proceso de obtención y preparación del recurso según corresponda. Además, las fuentes que generan ficheros en la obtención y preparación del recurso, reciben una ruta donde almacenarlos.

3. **ResourceLocator**: Esta clase se encarga de gestionar el catálogo de recursos del sistema. El catálogo cuenta con una sección para modelos y otra para *datasets* en las que guardará el nombre, *path*, estado del recurso, distinguiendo entre recurso instalado y no instalado, y si es manejado por el sistema. El método `resolve` se encarga de obtener y preparar un recurso nuevo haciendo uso del `ResourceSourceResolver` o, en el caso de ser un recurso previamente usado, devolver su localización en disco, delegando de nuevo en el `ResourceSourceResolver` en caso de ser necesario obtenerlo otra vez. El método `register` sirve para registrar modelos, útil para registrar los modelos resultantes del entrenamiento.
4. **LoaderResolver**: Al igual que con el otro resolver, se sigue el patrón estrategia para la carga del modelo y *dataset*. Se definen las abstracciones `ModelLoader` y `DatasetLoader` con los métodos `can_load` y `load` que usarán los correspondientes *resolvers* para cargar el recurso correspondiente. En el caso de los *datasets*, los *loaders* definen el atributo `REQUIRED_FEATURE_GROUPS` para especificar los grupos de características esperados para un tipo concreto de *dataset*.
5. **Wrappers**: `ModelWrapper` y `DatasetWrapper` son las clases que devuelven los métodos de load de los resolvers. Estas clases encapsulan los recursos para exponer una interfaz común al resto de operaciones del sistema sin que estas tengan que depender de la estructura en concreto del tipo de recurso.

En la Figura 6.4 se muestra el diagrama de secuencia de interacciones entre los principales componentes durante la carga de un recurso:

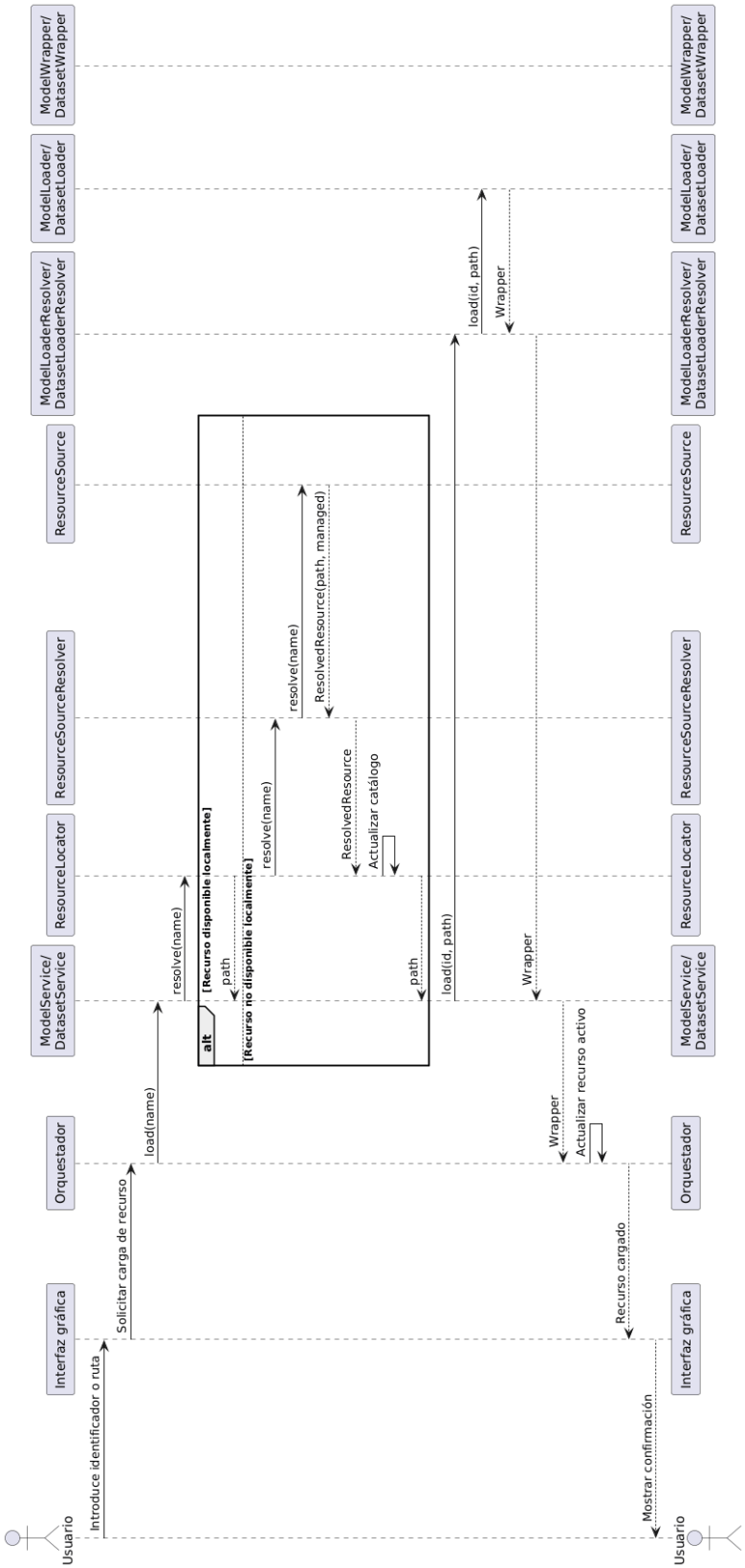


Figura 6.4: Diagrama de secuencia para carga de recursos

Como se puede observar en la Figura 6.4, el flujo comienza cuando el usuario solicita la carga de un recurso desde la interfaz gráfica. El orquestador delega la operación en el servicio correspondiente, que resuelve la localización del recurso, obteniéndolo de la fuente en caso de ser necesario. Una vez disponible localmente, el recurso se carga con el *loader* correspondiente y se encapsula en un *wrapper*.

Con este diseño, se facilita la posible incorporación de nuevas fuentes y tipos de modelos y *datasets* en el futuro, teniendo simplemente que implementar los métodos de la abstracción correspondiente sin la necesidad de cambiar el núcleo del sistema, cumpliendo de esta manera con el requisito RNF2 definido en la Sección 6.1.2.

6.3.3 Implementación

Para la implementación del **catálogo** se utiliza un fichero JSON dividido en secciones por recurso, teniendo de esta manera una sección para modelos y otra para *datasets*. Cada sección almacena los recursos utilizando como clave el identificador introducido por el usuario. Para cada uno se guarda su ruta en local, su estado, que podrá ser *downloaded* o *not_downloaded*, y si es gestionado por el sistema, es decir, si es un recurso generado por el propio sistema. Además, el sistema diferencia entre la desinstalación de un recurso, que elimina el recurso de local si está gestionado por el sistema y marca la entrada en el catálogo como no descargada, y la eliminación del recurso del catálogo, que elimina la referencia en el catálogo, además de desinstalarlo de local en caso de ser necesario. Finalmente, antes de devolver los recursos almacenados en el catálogo, el sistema comprueba que la ruta local asociada siga existiendo. De esta forma, si un recurso fue eliminado externamente al sistema, se actualiza su estado en el catálogo antes de devolver la lista de recursos disponibles.

Los **servicios** se crean mediante funciones *factory*, que permiten recibir opcionalmente la ruta del catálogo y el directorio base donde almacenar los recursos. En caso de no especificarse una ruta en concreto se usarán valores definidos por defecto. A partir del directorio base, el registro de *sources* instancia las fuentes con subdirectorios separados para modelos y *datasets*.

Con respecto a los ***sources***, en el *can_resolve* se realiza una comprobación ligera para determinar la fuente adecuada para resolver el recurso, mientras que en el *resolve* es donde se lleva a cabo la obtención y preparación del mismo. La resolución del recurso devuelve un `dataclass` con el *path* donde está guardado el recurso preparado y un indicador de si el recurso es gestionado por el sistema. Este será *False* para los recursos locales reutilizados directamente por el sistema, evitando así eliminar de local rutas externas al sistema. En el caso de los *sources* que descargan o preparan los recursos, recibirán también una ruta donde almacenarlos, usando una por defecto en caso de no especificarse una ruta en concreto.

En el caso de los **modelos**, simplemente se comprueba la existencia del *path* en local, y para recursos alojados en Hugging Face, se utiliza la librería `huggingface_hub`, concretamente el método `snapshot_download`, para descargar el repositorio del modelo y `repo_exists` para comprobar la existencia del modelo.

Por otra parte, los ***datasets*** se preparan en formato Arrow para facilitar la normalización de su estructura y la posterior detección del tipo del *dataset* en los *loaders*. De esta manera, tanto para local como para Hugging Face se utiliza la librería `datasets` mediante los métodos `load_dataset` y `save_to_disk`, contemplando en el caso de los *datasets* locales los formatos CSV, JSON, JSONL, PARQUET y el propio Arrow. El dataset se gestiona siempre como un directorio, permitiendo almacenar diferentes particiones o *splits* del dataset. En el caso de Hugging Face, `repo_exists` se utiliza indicando que el repositorio es un *dataset*.

En las fuentes que generan recursos gestionados por el sistema, el proceso se realiza primero sobre un directorio temporal. Si el proceso finaliza correctamente, el directorio se mueve a la ruta definitiva y en caso de que se produzca un error, se elimina el directorio temporal evitando dejar recursos incompletos o corruptos.

Para los ***loaders***, al igual que en los *sources*, en el `can_load` se realiza una comprobación ligera y en el `load` se carga el recurso. Para los **modelos de difusión** se valida que la estructura del modelo siga el formato esperado por la librería de `diffusers`, comprobando la existencia del fichero *model_index.json* y del atributo *_diffusers_version*. Para las arquitecturas concretas, se comprueba además que contengan el *pipeline* correspondiente en el atributo *_class_name*.

Durante la carga de los **modelos de difusión**, también se elige el tipo de dato de PyTorch según el dispositivo que se esté utilizando. Si el dispositivo es CUDA, se utiliza una precisión más baja para reducir el consumo de memoria y mejorar la velocidad de inferencia. En este caso, se utiliza `bfloat16` si el dispositivo lo admite, y `float16` si no. Si el dispositivo es una CPU, se mantiene `float32` para asegurar la compatibilidad. Esto mejora la eficiencia del sistema y cumple con el RNF4 definido en la Sección 6.1.2.

Para los ***datasets***, se comprueba que el fichero *dataset_info.json* contenga las columnas esperadas. En el caso del *dataset* texto-imagen, se espera que tenga al menos una columna para texto y otra para imagen. En caso de existir diferentes *splits*, se comprueba que todos ellos cumplan los grupos de características requeridos por el *loader*.

Por último, durante las operaciones de carga se diferencian los errores relativos a la resolución del recurso, los relativos al tipo de recurso y los relativos a la ejecución. Cuando ningún *source* puede resolver el recurso indicado, se lanza una excepción `ResourceNotFoundError`. Por otro lado, si el recurso existe pero falla su obtención o preparación, por ejemplo debido a problemas de autenticación, memoria o almacenamiento, se utiliza `ResourcePreparationError`. Cuando no existe ningún *loader* compatible, o cuando durante la carga se determina que el recurso no es válido o no está soportado por el sistema, se lanzan las excepciones `ModelNotSupportedError` o `DatasetNotSupportedError`. En estos casos, el servicio elimina automáticamente la referencia mediante el método *forget* y desinstala el recurso de local en caso de estar gestionado por el sistema, evitando mantener recursos no soportados en el catálogo. Por otro lado, los errores de carga asociados a limitaciones del entorno, como la falta de memoria, se representan mediante `ModelLoaderError` o `DatasetLoaderError`, y no provocan la eliminación del recurso del catálogo.

6.3.4 Pruebas

Para comprobar el comportamiento de los módulos de carga de modelos y *datasets*, así como la validación de los requisitos, se realizaron las pruebas unitarias que se recogen en la Tabla 6.7. Estas pruebas comprueban la correcta carga de modelos y datasets desde las diferentes fuentes, las distintas operaciones sobre el catálogo y la respuesta del sistema ante fuentes y recursos no soportados.

Tabla 6.7: Resultados de test de la primera iteración.

CU	Nombre	Descripción	Resultado esperado	Resultado obtenido
CU-02	Carga de modelo local	Se selecciona un modelo local válido.	El modelo se carga y se registra correctamente en el catálogo como recurso no gestionado.	Resultado esperado.
CU-02	Carga de modelo desde Hugging Face	Se indica un identificador válido de Hugging Face.	El modelo se descarga, se carga y se registra en el catálogo como recurso gestionado.	Resultado esperado.
CU-03	Carga de *dataset* local	Se selecciona un *dataset* local en formato JSONL.	El *dataset* se transforma a formato *Arrow* y se registra en el catálogo como recurso gestionado.	Resultado esperado.
CU-03	Carga de *dataset* desde Hugging Face	Se indica un identificador válido de *dataset* en Hugging Face.	El *dataset* se prepara correctamente, se carga y se registra en el catálogo como recurso gestionado.	Resultado esperado.
CU-04	Desinstalación de recurso	Se desinstala un recurso previamente registrado.	El recurso queda registrado como no instalado en el catálogo. En caso de ser un recurso gestionado por el sistema, también se elimina de local.	Resultado esperado.
CU-05	Eliminación de recurso	Se elimina completamente un recurso del catálogo.	El recurso desaparece del catálogo. En caso de ser un recurso gestionado por el sistema, también se elimina de local.	Resultado esperado.

. . . . . *(continúa en la página siguiente)* . . . . . .

Tabla 6.7 – *(viene de la página anterior)*

CU	Nombre	Descripción	Resultado esperado	Resultado obtenido
CU-06	Limpieza de catálogo	Se eliminan todos los recursos del catálogo.	El catálogo queda vacío. Los recursos gestionados por el sistema se eliminan de local, mientras que los no gestionados permanecen en su ubicación original.	Resultado esperado.
CU-02 y CU-03	Fuente no resoluble	Se intenta cargar un recurso cuya fuente no puede ser resuelta por ningún *source*.	El sistema rechaza la operación indicando que no existe una fuente capaz de resolver el recurso.	Resultado esperado.
CU-02 y CU-03	Recurso no compatible	Se intenta cargar un recurso que ha sido resuelto correctamente, pero que no es compatible con ningún *loader*.	El sistema rechaza la operación indicando que el recurso no es compatible con los tipos soportados por el sistema, además de registrar el recurso en el catálogo y desinstalarlo en caso de ser un recurso gestionado por el sistema.	Resultado esperado.

Los resultados obtenidos tras la realización de las pruebas reflejadas en la Tabla 6.7 muestran el correcto funcionamiento de los módulos de carga de modelos y *datasets*.

6.4 Iteración 2: Inferencia

El objetivo de esta iteración es desarrollar el módulo de inferencia definido en la Sección 6.2, cumpliendo con el requisito funcional RF3. Dicho módulo se encarga de ejecutar la inferencia sobre modelos generativos previamente cargados en el sistema, permitiendo obtener salidas a partir de las entradas proporcionadas por el usuario. Al trabajar sobre modelos ya cargados, el módulo no realiza directamente la resolución ni la carga de recursos, manteniendo una separación de responsabilidades con el módulo de carga y siguiendo el requisito no funcional RNF1. Además, para mantener la extensibilidad definida en el requisito RNF2, se desacopla la definición de las entradas, parámetros y salidas de inferencia de las implementaciones concretas de cada modelo, facilitando así la futura incorporación de nuevos tipos de modelos sin modificar el núcleo del sistema.

6.4.1 Análisis

Para el diagrama de casos de uso se dispone nuevamente del actor **Usuario** y, como único caso de uso, la realización de inferencia sobre un modelo previamente cargado en el sistema. El diagrama se muestra en la Figura 6.5.

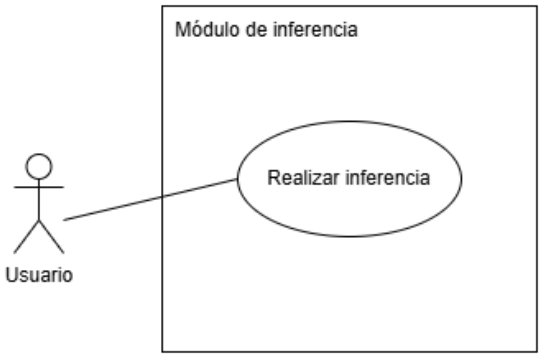


Figura 6.5: Casos de uso para iteración 2

En la Tabla 6.8 se muestra el único caso de uso de esta iteración, en donde se muestra una descripción, las precondiciones, la secuencia normal y las postcondiciones:

CU-07: Realizar inferencia	
Descripción	Permite realizar inferencia sobre un modelo previamente cargado, proporcionando los datos de entrada necesarios.
Precondiciones	El modelo debe estar cargado en el sistema.
Secuencia normal	1. El usuario accede al módulo de inferencia. 2. El sistema muestra las entradas y los parámetros de inferencia correspondientes. 3. El usuario introduce los datos de entrada y configura los parámetros necesarios. 4. El sistema ejecuta la inferencia sobre el modelo activo. 5. El sistema muestra el resultado obtenido.
Postcondiciones	El sistema muestra la salida generada al usuario.

Tabla 6.8: Caso de uso CU-07: Realizar inferencia

6.4.2 Diseño

Para el diseño del módulo de inferencia se diseñó la siguiente arquitectura, mostrada en la Figura 6.6:

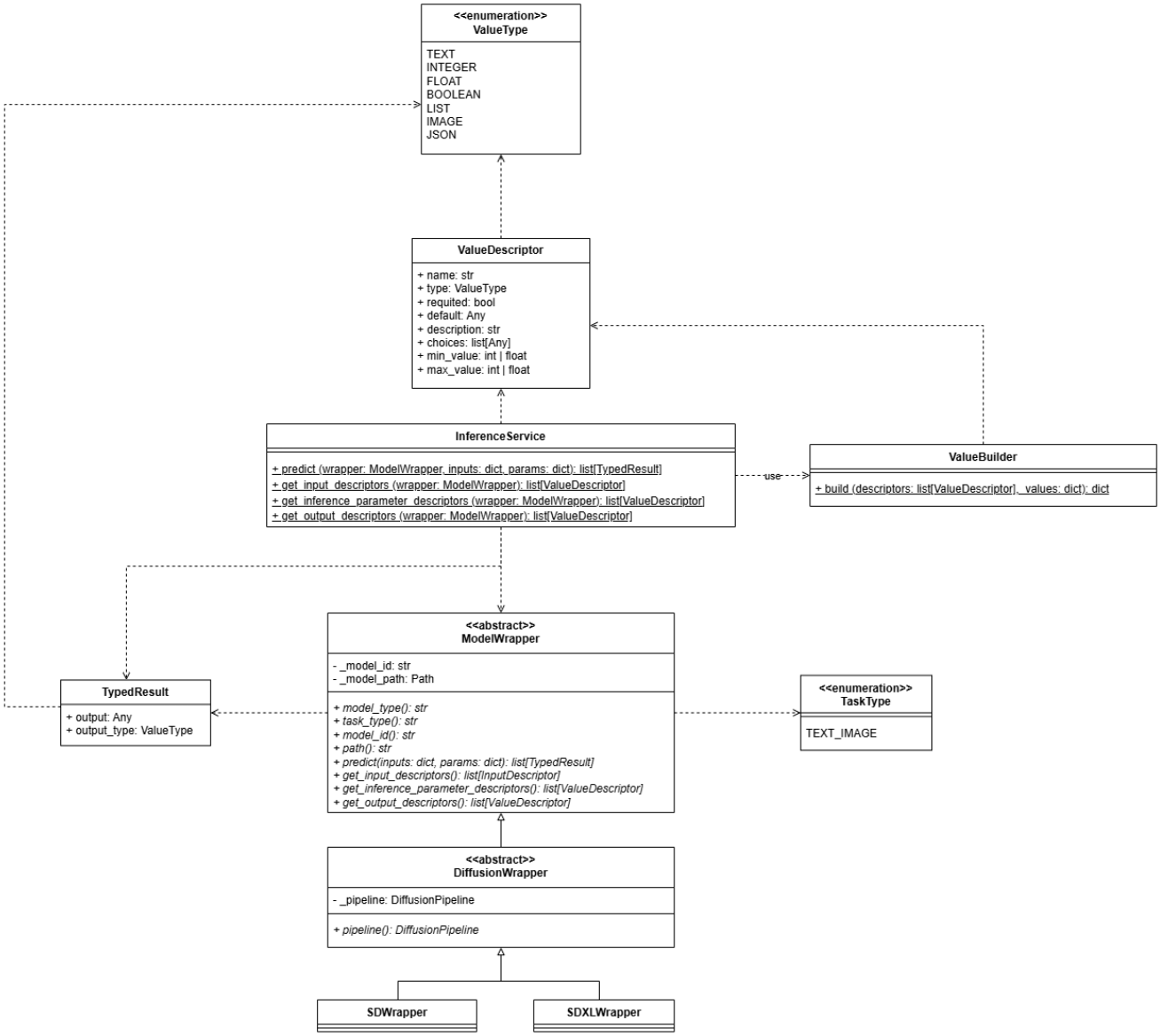


Figura 6.6: Diseño módulo de inferencia

En la Figura 6.6 se muestran los siguientes componentes:

1. **InferenceService**: El servicio dispone de los métodos `get_input_descriptors`, `get_inference_parameter_descriptors` y `get_output_descriptors`, encargados de obtener los descriptores de las entradas, parámetros y salidas utilizados

durante la inferencia sobre un modelo. Por otra parte, el método `predict` permite ejecutar la inferencia del modelo con las entradas y parámetros proporcionados.

2. **ModelWrapper**: La abstracción de los modelos cargados en el sistema, introducida en la iteración anterior, se amplía con los métodos necesarios para la inferencia. Para ello, se guarda su identificador y localización en local y se añaden los métodos `predict` y `get_input_descriptors`, `get_inference_parameter_descriptors` y `get_output_descriptors` encargados de ejecutar la inferencia y de describir las entradas, parámetros y salidas soportados por el modelo, respectivamente. Además, cuentan con los métodos de `model_type` y `task_type` que define cada modelo en concreto.

3. **ValueDescriptor**: Encapsula la información sobre las entradas, parámetros y salidas de la inferencia, exponiendo diferentes características como el nombre, tipo, obligatoriedad, valor por defecto, conjunto de opciones válidas, valores mínimo y máximo y una descripción. Cada modelo concreto devolverá sus listas de los descriptores correspondientes.

4. **ValueBuilder**: Se encarga de construir y validar los argumentos de inferencia a partir de los descriptores y de los valores proporcionados por el usuario.

5. **ValueType**: Enumerado que define los tipos de valores usados en los descriptores y en los resultados.

En la Figura 6.7 se muestra el diagrama de secuencia de interacciones entre los principales componentes durante la inferencia de un modelo previamente cargado en el sistema:

Como se puede observar en la Figura 6.7, la secuencia asume que existe un modelo cargado en el sistema. El flujo comienza cuando el usuario accede al módulo de inferencia. La interfaz gráfica solicita al servicio los descriptores de las entradas, parámetros y salidas para construir el formulario donde el usuario introducirá los valores correspondientes. Una vez introducidos los valores, el servicio delega su validación y construcción en el *builder*. A continuación, se ejecuta la inferencia, devolviendo los resultados para mostrar en la interfaz gráfica.

Con este diseño se mantiene, por una parte, la modularidad del sistema, al tratarse de un módulo completamente independiente del módulo de carga al recibir simplemente el *ModelWrapper* previamente cargado. Por otra parte, se favorece la extensibilidad, al permitir que cada modelo implemente su propia lógica de inferencia y exponga sus descriptores específicos según el modelo en concreto. De esta manera, se cumplen los requisitos RNF1 y RNF2 definidos en la Sección 6.1.2.

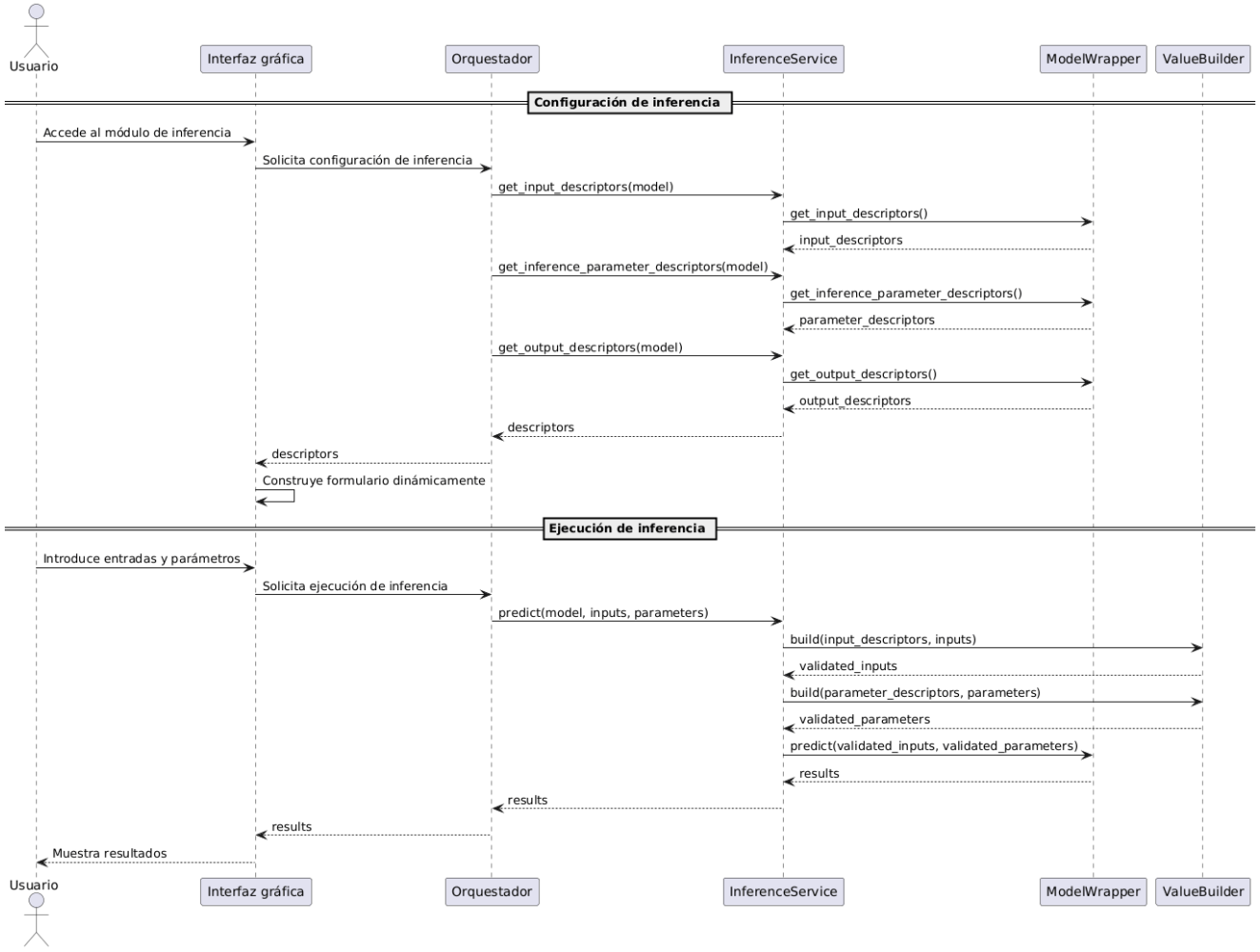


Figura 6.7: Diagrama de secuencia para inferencia de modelo

6.4.3 Implementación

Con respecto a la implementación, el *DiffusionWrapper* expone en los métodos `get_input_descriptors`, `get_inference_parameter_descriptors` y `get_output_descriptors` los descriptores comunes a los modelos de difusión de texto a imagen. Por una parte, como entradas el *prompt* principal y el *negative prompt* con carácter opcional. Por otra parte, los parámetros consisten en *steps* para el número de pasos de la inferencia, *guidance_scale* para el peso del condicionamiento textual durante la generación, el *seed* para la reproducibilidad y *num_images_per_prompt* para el número de imágenes por *prompt*. Con respecto a las salidas, está la lista de imágenes generadas. Los descriptores pueden incluir valores por defecto utilizados cuando el usuario no proporciona un valor, valores mínimo y máximo, y una descripción. Para el `SDXLWrapper`, se incluye además como entrada un segundo *prompt* para el segundo *text encoder* propio de esta arquitectura.

Para la implementación del método `predict` de los *wrappers*, se utilizan las entradas y parámetros correspondientes delegando el comportamiento en el pipeline del *wrapper*. Se utiliza la librería `torch` para crear el *generator* en caso de tener un *seed* diferente a -1 y también para establecer la opción de `inference_mode` para mejorar la eficiencia y cumplir con el requisito RNF4 definido en la Sección 6.1.2. El resultado de la inferencia se encapsula en `TypedResult`, devolviendo una lista de `TypedResult` con la imagen (o imágenes) generada y el tipo *image*.

La construcción de las entradas y los parámetros de la inferencia se realiza en el método `predict` de `InferenceService` previo a delegar la inferencia en el *wrapper* mediante el método `build` de `ValueBuilder`. Este método prepara los argumentos comprobando que todos los valores requeridos han sido proporcionados por el usuario, lanzando la excepción `InvalidValueError` cuando falta un valor obligatorio, cuando un valor no se encuentra dentro del rango mínimo y máximo definido, o cuando no pertenece al conjunto de opciones válidas. Además, realiza la conversión explícita al tipo indicado por cada descriptor y asigna los valores por defecto a las entradas y parámetros opcionales. De este modo, el método `predict` de los *wrappers* recibe argumentos ya validados y tipados.

6.4.4 Pruebas

Para comprobar el comportamiento del módulo de inferencia, así como la validación de los requisitos, se realizaron las pruebas unitarias que se recogen en la Tabla 6.9, donde se prueba las operaciones de inferencia, obtención de los descriptores del modelo y la preparación interna de las entradas y parámetros antes de delegar la ejecución en el *wrapper*. Además, se realizan

pruebas correspondientes a la eficiencia de la inferencia.

Tabla 6.9: Resultados de test de la segunda iteración.

CU	Nombre	Descripción	Resultado esperado	Resultado obtenido
CU-07	Obtención de descriptores de inferencia	Se solicita la lista de descriptores de un modelo compatible.	El sistema devuelve correctamente los descriptores de entradas, parámetros y salidas soportados por el modelo.	Resultado esperado.
CU-07	Preparación de argumentos de inferencia	Se ejecuta una inferencia utilizando un *wrapper* de prueba y valores introducidos por el usuario.	El sistema prepara correctamente las entradas y parámetros, aplicando conversiones y valores por defecto antes de delegar la ejecución en el *wrapper*.	Resultado esperado.
CU-07	Valor requerido ausente	Se intenta ejecutar una inferencia sin proporcionar una entrada requerida.	El sistema rechaza la operación lanzando la excepción correspondiente.	Resultado esperado.
CU-07	Inferencia con Stable Diffusion	Se ejecuta una inferencia sobre un modelo Stable Diffusion cargado, proporcionando entradas y parámetros válidos.	El sistema genera correctamente una imagen a partir del modelo.	Resultado esperado.
CU-07	Inferencia con Stable Diffusion XL	Se ejecuta una inferencia sobre un modelo SDXL cargado, proporcionando entradas y parámetros válidos, incluyendo el segundo *prompt*.	El sistema genera correctamente una imagen utilizando el modelo SDXL.	Resultado esperado.
CU-07	Tiempo de inferencia	Se mide el tiempo necesario para realizar una inferencia sobre un modelo previamente cargado.	Los tiempos son coherentes con la complejidad y características del modelo en concreto.	Resultado esperado.

Los resultados obtenidos tras la realización de las pruebas reflejadas en la Tabla 6.9 muestran

el correcto funcionamiento del módulo de inferencia.

6.5 Iteración 3: Procesamiento de entradas y salidas

El objetivo de esta iteración es desarrollar el módulo de procesamiento definido en la Sección 6.2, cumpliendo con el requisito funcional RF4. Dicho módulo permite aplicar operaciones de procesamiento sobre las entradas y salidas utilizadas durante la inferencia de un modelo. Estas operaciones pueden ejecutarse tanto antes como después de la inferencia, manteniendo una separación de responsabilidades entre ambos módulos, de forma que el módulo de inferencia recibe las entradas ya procesadas y las salidas generadas pueden ser enviadas posteriormente al módulo de procesamiento. De esta manera se sigue con el requisito RNF1. Además, para mantener la extensibilidad definida en el requisito RNF2, se desacopla la definición de las operaciones de procesamiento de sus implementaciones concretas, facilitando así la futura incorporación de nuevos procesadores sin necesidad de modificar el núcleo del sistema.

6.5.1 Análisis

Para el diagrama de casos de uso se dispone nuevamente del actor **Usuario**. Como caso de uso, se añade la visualización del procesamiento aplicado durante la inferencia. Además, el caso de uso *CU-07: Realizar inferencia*, explicado en la Sección 6.4.1, se amplía permitiendo el uso opcional de procesadores, de forma que el caso de uso *Visualizar procesamiento* extiende a *Realizar inferencia*. El diagrama se muestra en la Figura 6.8.

En la Tabla 6.10 se muestra la actualización del caso de uso *CU-07: Realizar inferencia*, y en la Tabla 6.11 el nuevo caso de uso, mostrando una descripción, las precondiciones, la secuencia normal y las postcondiciones.

6.5.2 Diseño

Para el diseño del módulo de procesamiento se diseñó la siguiente arquitectura, mostrada en la Figura 6.9.

En la Figura 6.9 se muestran los siguientes componentes:

1. **ProcessingService**: El servicio dispone de los métodos `get_available_processors` para obtener la lista de procesadores disponibles para un tipo de dato concreto y `process` para aplicar secuencialmente una lista de procesadores sobre un valor específico, devolviendo un `Processing_Trace`, que contiene la entrada original y una lista de `ProcessorStepResult`, almacenando el identificador y nombre del

CU-07: Realizar inferencia	
Descripción	Permite realizar inferencia sobre un modelo previamente cargado, proporcionando los datos de entrada necesarios y aplicando opcionalmente procesadores sobre las entradas proporcionadas y salidas generadas antes y después de la inferencia.
Precondiciones	El modelo debe estar cargado en el sistema.
Secuencia normal	1. El usuario accede al módulo de inferencia. 2. El sistema muestra las entradas, los parámetros de inferencia y los procesadores disponibles para los tipos de dato de entrada y salida del modelo. 3. El usuario introduce los datos de entrada y configura los parámetros necesarios. 4. El usuario selecciona opcionalmente uno o varios procesadores de entrada y salida. 5. El sistema aplica los procesadores de entrada seleccionados. 6. El sistema ejecuta la inferencia sobre el modelo activo. 7. El sistema aplica los procesadores de salida seleccionados. 8. El sistema muestra el resultado obtenido.
Extensiones	8a. El usuario solicita visualizar el procesamiento aplicado. 8a.1. El sistema muestra los resultados parciales generados por los procesadores aplicados.
Postcondiciones	El sistema muestra la salida generada en la inferencia.

Tabla 6.10: Caso de uso CU-07 con procesamiento: Realizar inferencia

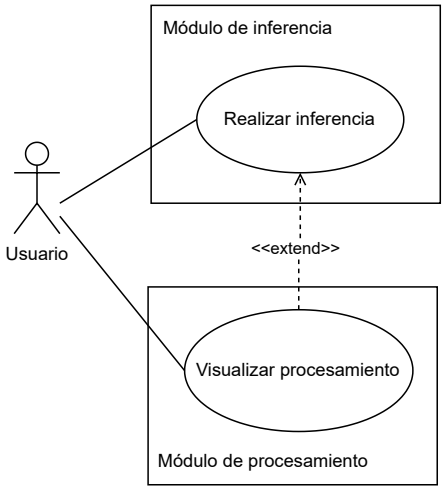


Figura 6.8: Casos de uso para iteración 3

CU-08: Visualizar procesamiento	
Descripción	Permite visualizar las transformaciones aplicadas sobre las entradas y salidas durante el proceso de inferencia.
Precondiciones	Se debe haber realizado previamente una inferencia.
Secuencia normal	1. El usuario solicita visualizar el procesamiento aplicado. 2. El sistema muestra los resultados parciales generados por las transformaciones aplicadas.
Postcondiciones	El usuario puede consultar las transformaciones aplicadas durante la inferencia.

Tabla 6.11: Caso de uso CU-08: Visualizar procesamiento

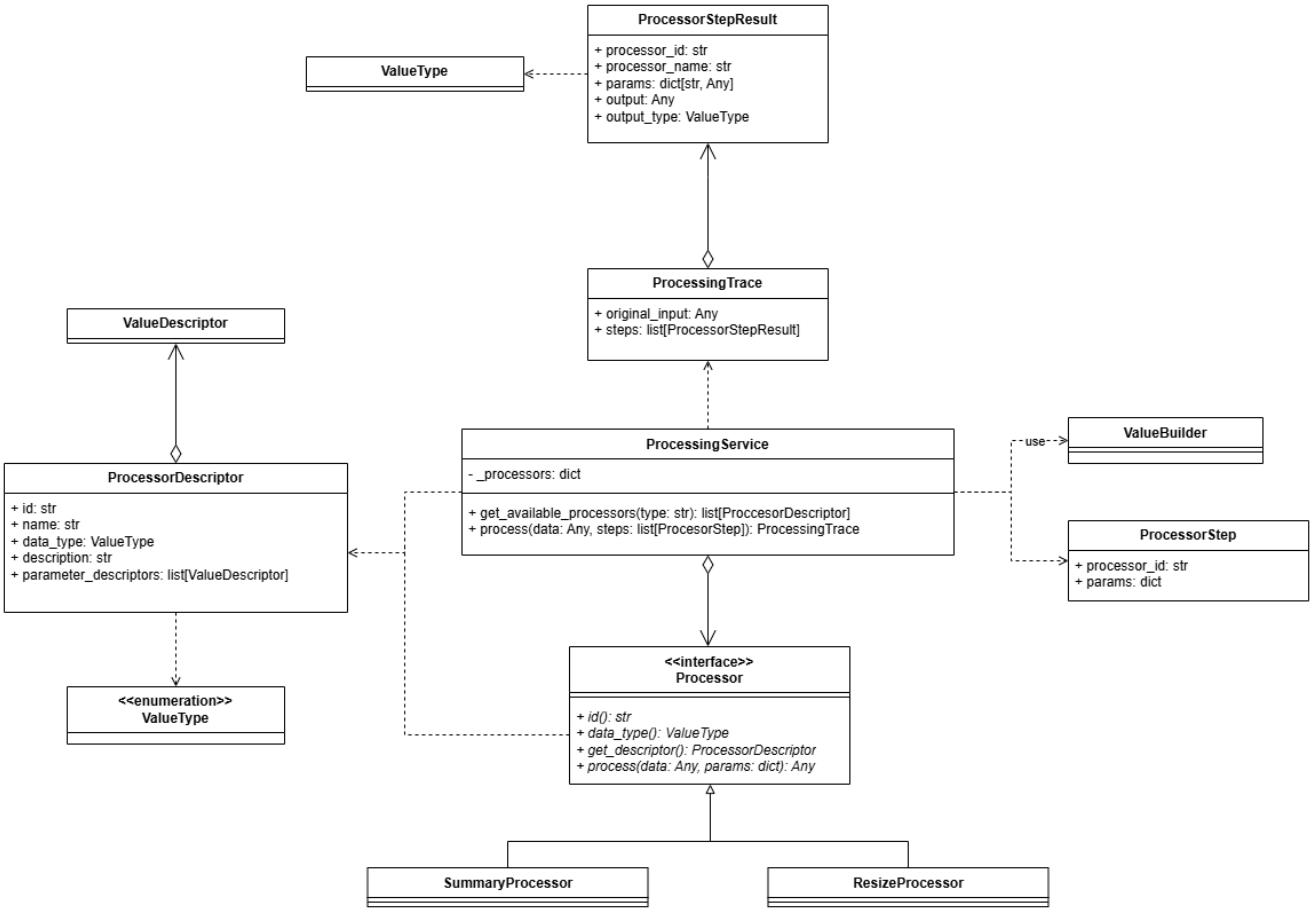


Figura 6.9: Diseño módulo de procesamiento

procesador, parámetros utilizados y la salida junto con su tipo. El *service* hace uso del `ValueBuilder` para validar y construir los parámetros de cada procesador, además de establecer valores por defecto cuando sea necesario.

2. **Processor**: Cada procesador cuenta con un identificador utilizado por el *service* para recuperar el procesador correspondiente, tipo de dato asociado y con los métodos `get_descriptor`, que proporciona información del procesador mediante el `ProcessorDescriptor`, y `process`, que realiza el procesamiento sobre un valor de entrada a partir de los parámetros proporcionados. Se restringe a transformaciones que mantienen el mismo tipo de dato antes y después del procesamiento, simplificando así la composición secuencial de las transformaciones y evitando tener que hacer comprobaciones adicionales de compatibilidad. Como implementaciones concretas se encuentran `SummarizeProcessor` y `ResizeProcessor` para resumir texto y reescalar una imagen, respectivamente.

3. **ProcessorStep**: Representa un paso a aplicar en la secuencia de procesadores, indicando el identificador del procesador y los parámetros en caso de ser necesario. De esta forma se permite tener varias veces el mismo procesador pero con diferentes parámetros.

4. **ProcessorDescriptor**: Encapsula información sobre el procesador como su identificador, nombre, tipo de dato asociado, una descripción y los descriptores de parámetros del procesador necesarios para su configuración.

En la Figura 6.10 se muestra el diagrama de secuencia de interacciones entre los principales componentes durante el procesamiento de entradas y salidas, ampliando el diagrama mostrado en la Figura 6.7 sobre la inferencia:

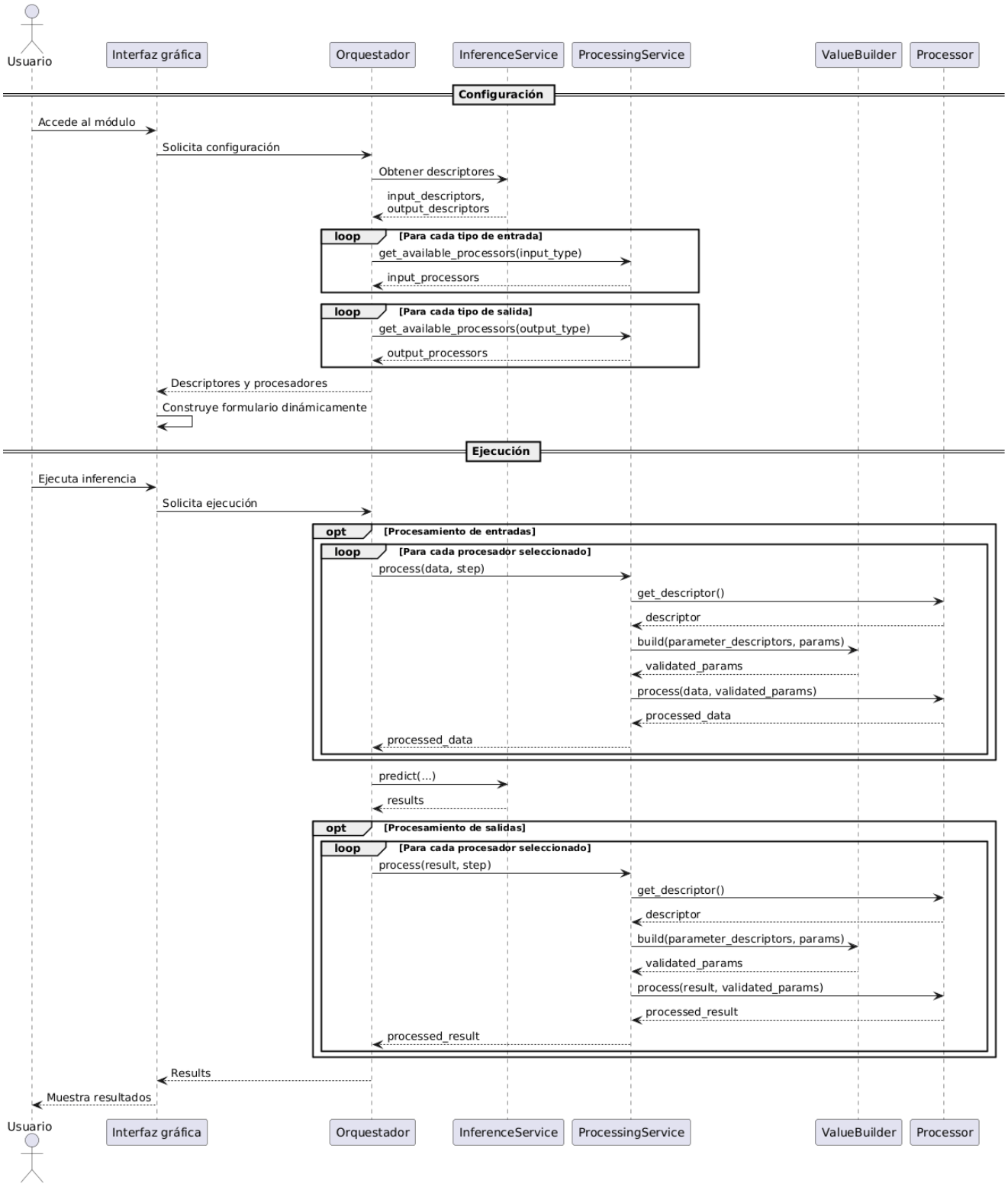


Figura 6.10: Diagrama de secuencia para procesamiento

Como se puede observar en la Figura 6.10, la secuencia asume que existe un modelo cargado en el sistema. El flujo comienza cuando el usuario accede al módulo de inferencia. A partir de los descriptores de las entradas y salidas obtenidos por el servicio de inferencia (ver detalles en la Figura 6.7), el orquestador obtiene los descriptores de los procesadores disponibles para cada tipo a través del servicio de procesamiento. Una vez solicitada la operación de inferencia, se aplica la cadena de procesadores seleccionada sobre las entradas, construyendo sus parámetros mediante el *builder* cuando corresponda. El resultado del procesamiento se devuelve al orquestador, que realiza la inferencia. El resultado de la inferencia se envía de vuelta al orquestador, que procesa las salidas mediante el servicio de procesamiento, de la misma forma que se hizo con las entradas. Finalmente, se envían tanto las entradas como salidas procesadas a la interfaz gráfica.

Con este diseño se mantiene nuevamente la modularidad del sistema, al conservar el módulo de inferencia desacoplado del de procesamiento, de forma que las transformaciones de datos puedan aplicarse antes o después de la inferencia sin acoplar ambos módulos. Además, cada procesador cuenta con su propia lógica de procesamiento y sus parámetros necesarios, favoreciendo así la extensibilidad del sistema y permitiendo la posible incorporación de nuevos procesadores en el futuro. De esta manera se cumple con los requisitos RNF1 y RNF2 definidos en la Sección 6.1.2.

6.5.3 Implementación

Para la implementación, el servicio recibe la lista de los procesadores disponibles en el sistema y mantiene dos diccionarios internos para su utilización, de forma que uno indexa por identificador y el otro por *data_type*. De esta manera, se permite encontrar de manera más eficiente el procesador por identificador y también la lista de procesadores disponibles para un tipo de dato en concreto.

Para el método `process`, se recibe el dato y una lista de `ProcessorStep`, donde cada paso contiene el identificador del procesador y opcionalmente sus parámetros asociados. Primero se resuelven los identificadores de procesadores y se valida la compatibilidad de la cadena. A continuación, se aplican las transformaciones secuencialmente guardando los resultados en una lista de `TypedResult`. Para la construcción de los parámetros se hace uso del `ValueBuilder`.

Antes de aplicar la cadena de procesadores, el servicio valida que todos los identificadores correspondan con procesadores registrados y que todos tengan el mismo *data_type*. Si el identificador no existe, se lanza una excepción `ProcessorNotFoundError` y, si la

cadena no comparte tipo de dato o los parámetros no cumplen el formato esperado, se lanza una excepción `InvalidValueError`. Por otra parte, si algún procesador falla durante su ejecución interna, se lanza una excepción `ProcessorError`.

Con respecto a las implementaciones concretas, en el caso de `SummarizeProcessor` se establece el identificador `summarize` y `text` como tipo de dato. Para el procesamiento, se hace uso del modelo preentrenado `sshleifer/distilbart-cnn-12-6` [41] de Hugging Face entrenado para tareas de resumen de texto y de las librerías de `transformers` y `torch`. El texto de entrada se tokeniza en formato tensorial compatible con PyTorch y se establece un máximo de 1024 tokens, habilitando la opción de truncado en caso de exceder el límite. Para la eficiencia, el procesador selecciona automáticamente GPU mediante *cuda* en caso de estar disponible, se establece el modelo en modo evaluación con `eval` y se ejecuta el modelo con la opción `inference_mode` para evitar el cálculo de gradientes durante la inferencia. Además, una vez finalizado el procesamiento, se libera explícitamente el modelo, el tokenizador y los tensores generados, además de vaciar la caché de CUDA, para reducir el consumo de memoria en el uso posterior del sistema. De esta forma, se sigue cumpliendo nuevamente con el requisito RNF4 definido en la Sección 6.1.2.

El `ResizeProcessor` permite realizar la operación de redimensionado sobre una imagen. Se establece el identificador *resize_image* e *image* como tipo de dato. Se utiliza el método de remuestreo de *Lanczos* para el redimensionado de imagen, reduciendo pérdida de detalle frente a métodos más simples como *nearest* o *bilinear* de la librería `Pillow` [29]. Este procesador cuenta con los parámetros de anchura y altura de imagen con unos valores mínimos, máximos y por defecto para limitar tamaños excesivos y controlar el consumo de memoria.

6.5.4 Pruebas

Para comprobar el comportamiento del módulo de procesamiento se realizaron las pruebas unitarias que se recogen en la Tabla 6.12, donde se comprueba la obtención de los procesadores disponibles para un tipo de dato, la ejecución secuencial de los procesadores y la respuesta del sistema ante procesadores inexistentes y procesadores incompatibles.

Tabla 6.12: Resultados de test de la tercera iteración.

CU	Nombre	Descripción	Resultado esperado	Resultado obtenido
CU-07	Obtención de procesadores disponibles	Se solicita la lista de procesadores disponibles para un tipo de dato en concreto.	El sistema devuelve correctamente los procesadores disponibles para el tipo de dato.	Resultado esperado.
CU-07	Procesamiento con *SummarizeProcessor*	Se realiza el procesamiento de resumen de texto.	El sistema realiza correctamente el procesamiento y resume el texto de entrada.	Resultado esperado.
CU-07	Procesamiento con *ResizeProcessor*	Se realiza el procesamiento de redimensionado de imágenes.	El sistema realiza correctamente el procesamiento y redimensionado las imágenes con el tamaño solicitado.	Resultado esperado.
CU-07	Procesador inexistente	Se solicita el procesamiento con un identificador inexistente.	El sistema rechaza la operación indicando que no existe el procesador solicitado.	Resultado esperado.
CU-07	Procesamiento con una cadena incompatible de procesadores	Se solicita el procesamiento con procesadores con diferente tipo de dato.	El sistema rechaza la operación indicando que los procesadores esperan diferentes tipos de datos.	Resultado esperado.

Los resultados obtenidos tras la realización de las pruebas mostradas en la Tabla 6.12 muestran el correcto funcionamiento del módulo de procesamiento.

6.6 Iteración 4: Ajuste fino

El objetivo de esta iteración es desarrollar el módulo de ajuste fino definido en la Sección 6.2, cumpliendo con los requisitos funcionales RF5, RF6 y RF7. Para ello, se implementa un módulo específico para el ajuste fino, siguiendo el requisito no funcional RNF1. Dicho módulo permite realizar procesos de ajuste fino sobre modelos y *datasets* previamente cargados utilizando diferentes técnicas de entrenamiento. Además, se gestiona un catálogo de entrenamientos y un mecanismo de guardado de puntos de control para permitir al usuario reanudar un entrenamiento. El catálogo muestra información básica sobre el entrenamiento como identificadores del entrenamiento, modelo y *dataset*, *split* de *dataset*, técnica de ajuste fino, hiperparámetros

utilizados, última época con *checkpoint* registrado y estado del entrenamiento, que puede ser en ejecución, completado, cancelado o fallido. Este módulo reutiliza las abstracciones definidas en los módulos de carga de modelos y *datasets*, permitiendo desacoplar las técnicas de ajuste fino de las implementaciones concretas de cada modelo. Además, para mantener la extensibilidad definida en el requisito RNF2, se desacopla la definición de los procesos de entrenamiento e hiperparámetros necesarios de las implementaciones concretas, facilitando así la futura incorporación de nuevas técnicas sin modificar el núcleo del sistema. Por último, se permite también el uso de adaptadores generados por el sistema durante el proceso de ajuste fino para la inferencia, desacoplando la carga y descarga de adaptadores del módulo de inferencia y también del tipo concreto de adaptador.

6.6.1 Análisis

En el diagrama de casos de uso se dispone al **Usuario** como actor. Como casos de uso, se añade la ejecución del entrenamiento sobre un modelo seleccionado con un *dataset* y técnica de ajuste fino compatibles, la reanudación de un entrenamiento guardado en el catálogo, la eliminación de un entrenamiento y la limpieza del catálogo. Además, el caso de uso *CU-07: Realizar inferencia*, explicado en la Sección 6.5.1, se amplía nuevamente permitiendo el uso opcional de adaptadores. El diagrama se muestra en la Figura 6.11

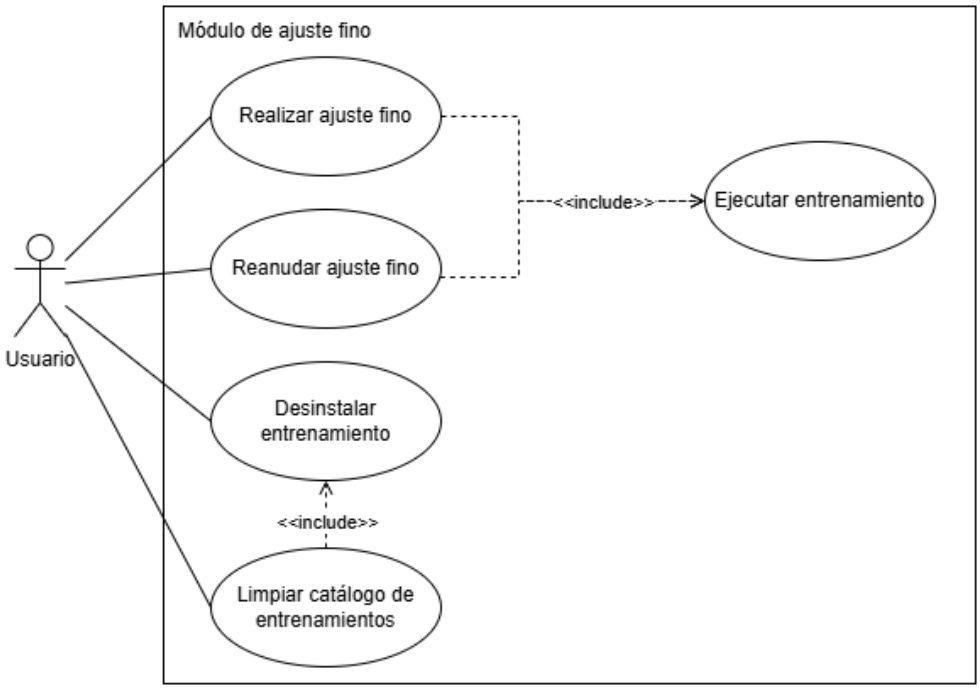


Figura 6.11: Casos de uso para iteración 4

En las Tablas 6.13 a 6.18 se muestran los casos de uso, mostrando una descripción, las precondiciones, la secuencia normal y las postcondiciones:

CU-07: Realizar inferencia	
Descripción	Permite realizar inferencia sobre un modelo previamente cargado, proporcionando los datos de entrada necesarios y aplicando opcionalmente procesadores sobre las entradas proporcionadas y salidas generadas antes y después de la inferencia.
Precondiciones	El modelo debe estar cargado en el sistema.
Secuencia normal	1. El usuario accede al módulo de inferencia. 2. El sistema muestra las entradas, los parámetros de inferencia y los procesadores disponibles para los tipos de dato de entrada y salida del modelo. 3. El usuario introduce opcionalmente uno o varios adaptadores compatibles con el modelo cargado. 4. El usuario selecciona opcionalmente uno o varios procesadores de entrada y salida. 5. El sistema aplica los procesadores de entrada seleccionados. 6. El sistema ejecuta la inferencia sobre el modelo activo. 7. El sistema aplica los procesadores de salida seleccionados. 8. El sistema muestra el resultado obtenido.
Extensiones	8a. El usuario solicita visualizar el procesamiento aplicado. 8a.1. El sistema muestra los resultados parciales generados por los procesadores aplicados.
Postcondiciones	El sistema muestra la salida generada en la inferencia.

Tabla 6.13: Caso de uso CU-07 con adaptadores: Realizar inferencia

CU-09: Realizar ajuste fino	
Descripción	Permite realizar el ajuste fino de un modelo previamente cargado, utilizando un *dataset* y una técnica compatibles.
Precondiciones	El modelo y el *dataset* deben estar cargados en el sistema y debe existir una técnica de ajuste fino compatible con ambos.
Secuencia normal	1. El usuario accede al módulo de ajuste fino. 2. El sistema muestra las técnicas compatibles con el modelo y el *dataset* cargados. 3. El usuario selecciona opcionalmente el *split* del *dataset* a utilizar en caso de disponer de varios. 4. El usuario selecciona opcionalmente el número de muestras aleatorias a utilizar del *dataset* 5. El usuario selecciona la técnica de ajuste fino. 6. El sistema muestra los parámetros de la técnica seleccionada. 7. El usuario configura los parámetros del ajuste fino. 8. El sistema ejecuta el caso de uso CU-11: Ejecutar entrenamiento.
Postcondiciones	El sistema genera y registra el recurso resultante del ajuste fino. Además, el catálogo de entrenamientos queda actualizado con el estado del entrenamiento.

Tabla 6.14: Caso de uso CU-09: Realizar ajuste fino

CU-10: Reanudar ajuste fino	
Descripción	Permite reanudar un entrenamiento de ajuste fino previamente registrado, utilizando el último punto de control disponible.
Precondiciones	Debe existir un entrenamiento registrado con un punto de control válido y los recursos asociados deben estar disponibles o poder cargarse desde el catálogo.
Secuencia normal	1. El usuario accede al módulo de ajuste fino. 2. El sistema muestra los entrenamientos registrados. 3. El usuario selecciona un entrenamiento reanudable. 4. El sistema carga el modelo y el *dataset* asociados al entrenamiento. 5. El sistema recupera la técnica, los hiperparámetros y el último punto de control registrados. 6. El sistema ejecuta el caso de uso CU-11: Ejecutar entrenamiento.
Postcondiciones	El entrenamiento queda actualizado en el catálogo y, en caso de finalizar correctamente, se genera el recurso resultante del ajuste fino.

Tabla 6.15: Caso de uso CU-10: Reanudar ajuste fino

CU-11: Ejecutar entrenamiento	
Descripción	Permite ejecutar un entrenamiento de ajuste fino utilizando una configuración previamente definida.
Precondiciones	Debe existir una técnica de ajuste fino configurada junto con el modelo y *dataset* asociados.
Secuencia normal	1. El sistema inicia el entrenamiento. 2. El sistema guarda puntos de control durante el entrenamiento cuando corresponda. 3. El sistema actualiza el catálogo de entrenamientos con el estado de la ejecución. 4. El sistema almacena y registra el recurso resultante.
Extensiones	1a. El usuario cancela el entrenamiento. 1a.1. El sistema detiene el entrenamiento y actualiza el catálogo con el estado de la ejecución. 1a.2. Si existe un punto de control válido, el entrenamiento queda disponible para ser reanudado posteriormente.
Postcondiciones	El catálogo de entrenamientos queda actualizado y, si el entrenamiento finaliza correctamente, se genera y registra el recurso resultante.

Tabla 6.16: Caso de uso CU-11: Ejecutar entrenamiento

CU-12: Eliminar entrenamiento	
Descripción	Permite eliminar un entrenamiento registrado junto con los ficheros asociados almacenados en disco, incluyendo checkpoints y resultados generados.
Precondiciones	Debe existir un entrenamiento registrado en el catálogo.
Secuencia normal	1. El usuario accede a la gestión de entrenamientos registrados. 2. El sistema muestra los entrenamientos registrados. 3. El usuario selecciona un entrenamiento. 4. El usuario solicita desinstalar el entrenamiento. 5. El sistema elimina los ficheros asociados al entrenamiento. 6. El sistema elimina la entrada del catálogo.
Postcondiciones	El entrenamiento es eliminado tanto del catálogo como de local.

Tabla 6.17: Caso de uso CU-12: Eliminar entrenamiento

CU-13: Limpiar catálogo de entrenamientos	
Descripción	Permite eliminar todos los entrenamientos registrados en el catálogo.
Precondiciones	El sistema debe estar operativo.
Secuencia normal	1. El usuario solicita limpiar el catálogo. 2. El sistema desinstala los ficheros asociados al entrenamiento. 3. El sistema elimina todas las entradas del catálogo.
Postcondiciones	El catálogo queda vacío y se eliminaron localmente todos los entrenamientos.

Tabla 6.18: Caso de uso CU-13: Limpiar catálogo de entrenamientos

6.6.2 Diseño

Para el diseño del módulo de ajuste fino se diseñó la siguiente arquitectura, mostrada en la Figura 6.12:

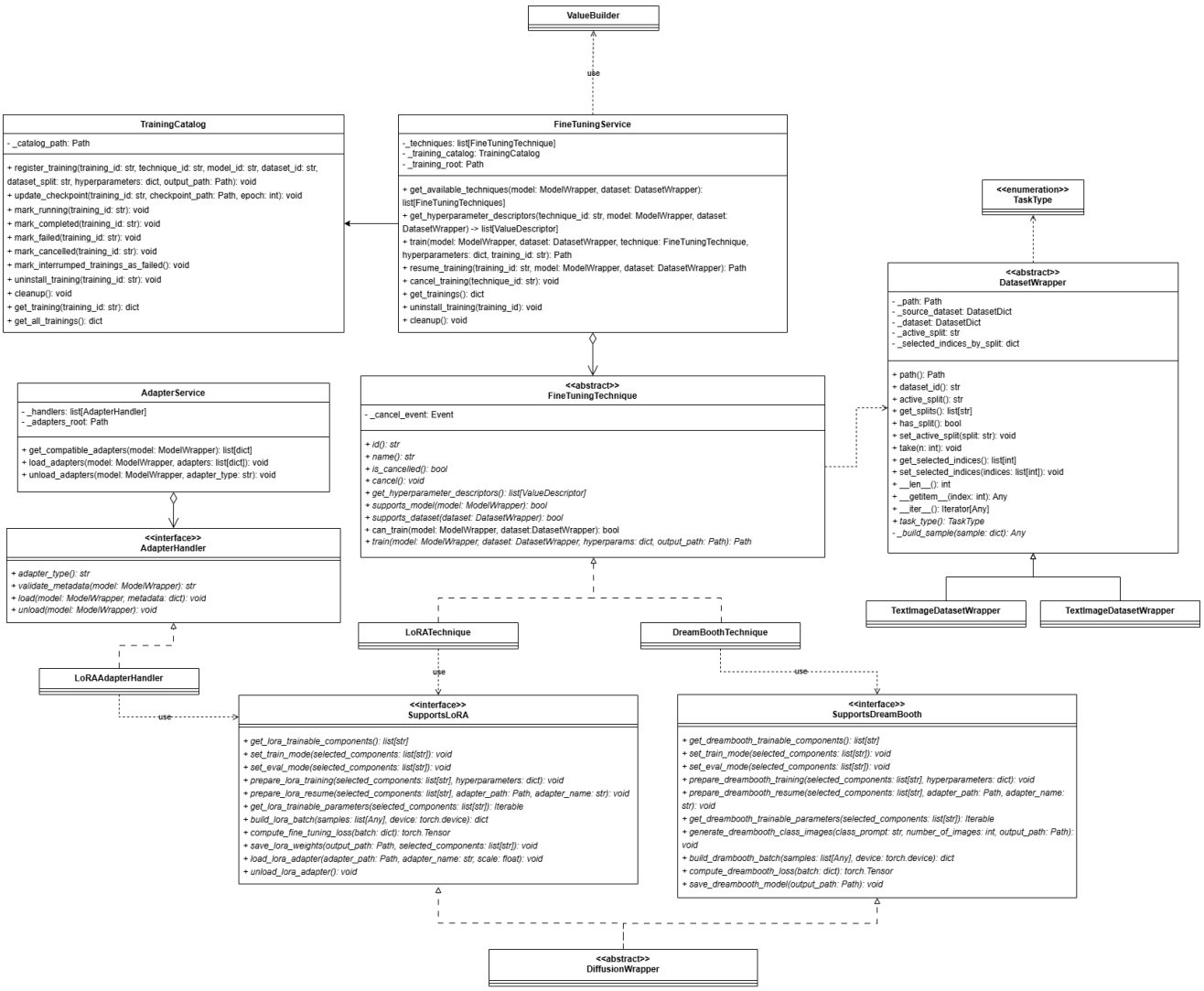


Figura 6.12: Diseño módulo de ajuste fino

En la Figura 6.12 se muestran los siguientes componentes:

1. **FineTuningService**: Por una parte, se encarga de obtener las técnicas compatibles dado un modelo y un *dataset* determinados y de iniciar, reanudar y cancelar un entrenamiento. Por otra parte, cuenta con métodos para la gestión del catálogo de entrenamientos, como `get_trainings` para obtener todos los entrenamientos, `uninstall_training` para eliminar un entrenamiento del almacenamiento local y del catálogo y `cleanup` para hacer la operación anterior con todos los entrenamientos registrados. Este servicio hace uso del `ValueBuilder` para validar y construir los hiperparámetros de la técnica, además de establecer valores por defecto cuando sea necesario.

2. **TrainingCatalog**: Esta clase se encarga de gestionar el catálogo de entrenamientos del

sistema. Para cada entrenamiento se almacena información necesaria para identificar el entrenamiento, los recursos y configuración utilizados, controlar su estado y permitir su reanudación. También cuenta con métodos para registrar, consultar, actualizar y eliminar entrenamientos.

3. **AdapterService**: Se encarga de cargar y descargar adaptadores para un modelo. Para facilitar la incorporación de nuevos tipos de adaptadores, se aplica el patrón **Estrategia** mediante la interfaz `AdapterHandler`, que define el comportamiento común con las operaciones de validación, carga y descarga. Además, el método `get_compatible_adapters` obtiene los adaptadores compatibles de un modelo determinado. Como implementación concreta se encuentra `LoRAAdapterHandler` que define el comportamiento específico para adaptadores LoRA.

4. **DatasetWrapper**: El *wrapper* cuenta con el *split* activo y diferentes métodos para manejar *datasets* como `DatasetDict`, con los métodos `len`, `getitem` y `iter` para convertir el *wrapper* en una colección iterable y `_build_sample`, que permite que cada tipo de *dataset* en concreto defina cómo transformar filas originales del *dataset* en muestras normalizadas del sistema. Además, se gestiona también la selección de subconjuntos de muestras. Como implementaciones concretas se encuentran `TextImageDatasetWrapper` y `ImageDatasetWrapper`.

5. **FineTuningTechnique**: Cada técnica de ajuste fino cuenta con un identificador utilizado por el *service* para recuperar la técnica correspondiente, el nombre, y los métodos `get_hyperparameter_descriptors` para obtener los hiperparámetros a configurar por el usuario, haciendo uso del `ValueDescriptor`, `supports_model` y `supports_dataset`, que comprueban que la técnica sea compatible con el modelo y *dataset* correspondientes, `can_train` que hace uso de los dos anteriores métodos y `train`, que se encarga de realizar el entrenamiento a partir de un modelo, un *dataset*, los hiperparámetros y almacena el resultado en `output_path`. Además, también están los métodos `cancel` y `is_cancelled` para la cancelación del entrenamiento. Como implementaciones concretas se encuentran las técnicas de LoRA y DreamBooth.

6. **SupportsLoRA y SupportsDreamBooth**: Protocolos que desacoplan las técnicas de ajuste fino de las arquitecturas concretas de los modelos. Ambos definen métodos que se necesitan para el entrenamiento, como obtener los componentes entrenables, establecer el modelo en modo entrenamiento o evaluación, preparar el modelo, construir los *batches*, calcular la pérdida y guardar el resultado. En el caso de `SupportsLoRA` se dispone de métodos para guardar, cargar y descargar adaptadores y, para `SupportsDreamBooth` se añaden operaciones para generar imágenes de clase y guardar el modelo completo.

En las Figuras 6.13 y 6.14 se muestran los diagramas de secuencia de interacciones entre los principales componentes durante el ajuste fino y reanudación de un entrenamiento, respectivamente:

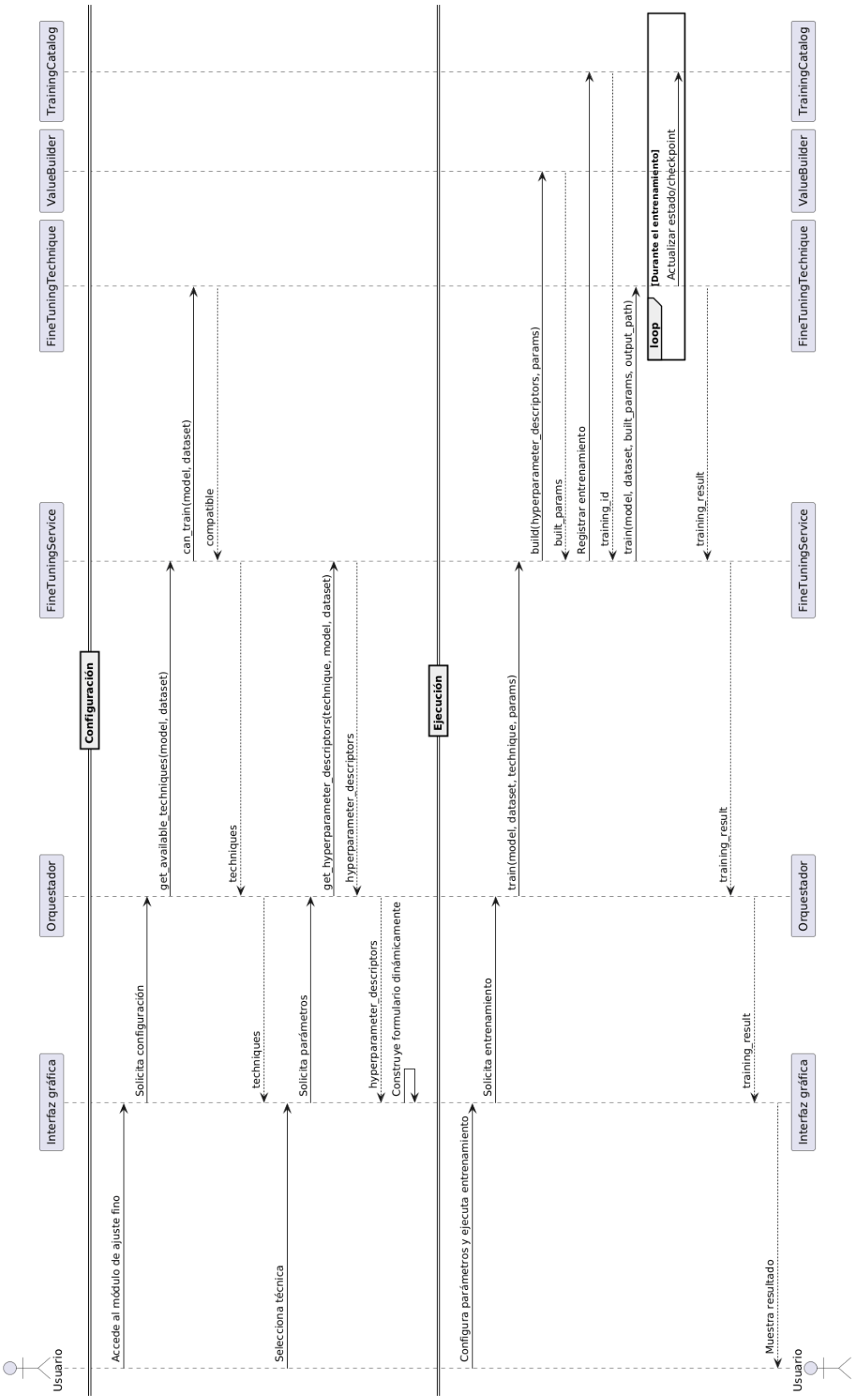


Figura 6.13: Diagrama de secuencia para ajuste fino

Como se puede observar en la Figura 6.13, la secuencia asume que existe un modelo y un *dataset* cargados en el sistema. El flujo comienza cuando el usuario accede al módulo de ajuste fino. El orquestador solicita al servicio las técnicas compatibles con el modelo y *dataset*. Una vez seleccionada la técnica se obtienen los descriptores de los hiperparámetros que usará la interfaz gráfica para construir el formulario. El usuario introduce los valores y se construyen con el *builder*. Finalmente, se realiza el ajuste fino delegando la ejecución en la técnica seleccionada, que se encarga de actualizar el catálogo y los puntos de control durante el entrenamiento.

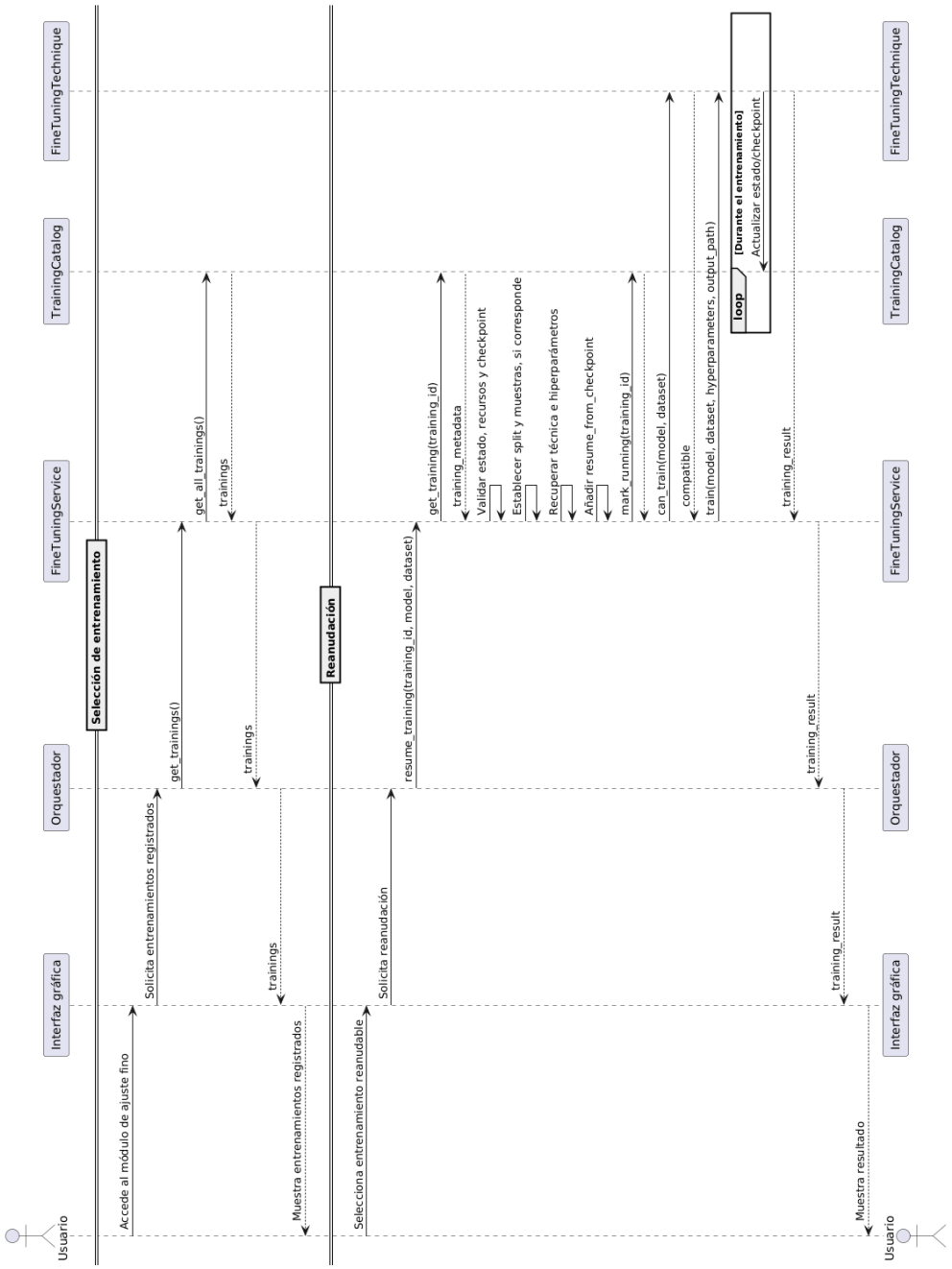


Figura 6.14: Diagrama de secuencia para reanudación de entrenamiento

Como se puede observar en la Figura 6.14, nuevamente la secuencia asume que existe un modelo y un *dataset* cargados en el sistema y el flujo comienza cuando el usuario accede al módulo de ajuste fino. El orquestador solicita al servicio los entrenamientos realizados y el usuario solicita uno reanudable. El servicio obtiene la información asociada al entrenamiento y valida que el estado sea correcto (cancelado o fallido) y los recursos coincidan, además de verificar la existencia de un punto de control válido para la reanudación. Además, establece el *split* y las muestras correspondientes. Finalmente, se continúa el entrenamiento, delegando la ejecución en la técnica seleccionada y actualizando el catálogo y los puntos de control.

Con este diseño se mantiene nuevamente la modularidad del sistema, al tener el módulo de ajuste fino desacoplado de la resolución y carga de modelos y de *datasets* de forma que el entrenamiento recibe un *ModelWrapper* y un *DatasetWrapper* previamente cargados. Al mismo tiempo, se mantienen desacoplados también el servicio de ajuste fino y el de adaptadores, contando con dos servicios diferentes. Por otra parte, se favorece la extensibilidad al disponer de una interfaz genérica de técnica de ajuste fino, lo que facilita la posible incorporación de nuevas técnicas en el futuro. Asimismo, los protocolos de `SupportsLoRA` y `SupportsDreamBooth` permiten desacoplar técnica de entrenamiento de la implementación concreta del modelo utilizando métodos que cada tipo de modelo debe implementar en función de sus características y especificaciones propias. Y, por último, también se favorece la extensibilidad mediante la definición de la interfaz para los adaptadores, de forma que se puedan añadir nuevos en el futuro sin modificar el núcleo del sistema. De esta forma se cumplen con los requisitos RNF1 y RNF2 definidos en la Sección 6.1.2.

6.6.3 Implementación

El **servicio de ajuste fino** recibe la lista de las técnicas disponibles en el sistema y mantiene un diccionario indexado por identificador. En el método `train` se comprueba que la técnica exista y que sea compatible con el modelo y *dataset* mediante el método `can_train`. Este método comprueba que `supports_model` y `supports_dataset` sean *true* y que el `task_type` de ambos sea el mismo. Además, si el identificador de entrenamiento ya existe se lanza una excepción de `TrainingAlreadyExistsError`. Para la construcción de los hiperparámetros se hace uso del `ValueBuilder`. Para el método `resume_training` se obtiene el entrenamiento del catálogo y se comprueba que sea reanudable (*status* con *cancelled* o *failed*) y que los *wrappers* correspondan con los identificadores registrados. Se establece el *split* correspondiente del *dataset* y en caso de haber establecido un subconjunto del *dataset*, se usan las muestras correspondientes. Por último, al crear el servicio se marcan como fallidos los entrenamientos en estado de ejecución, debido a que estos serían entrenamientos que quedaron en un estado inconsistente al cerrar la aplicación antes de poder finalizar correctamente.

Con respecto al **servicio de adaptadores**, se mantiene un diccionario interno indexado por tipo de adaptador, de forma que se busca por dicho tipo y se delega el comportamiento en los métodos definidos en las clases concretas de `AdapterHandler`. Para el adaptador de LoRA se comprueba que el modelo contenga el protocolo `SupportsLoRA` y que el modelo sea el mismo que el guardado en la metadata del adaptador. Los errores de metadata, inexistencia o incompatibilidad se representan mediante excepciones específicas. En `get_compatible_adapters` se buscan carpetas de `adapters_root` con un *metadata.json* que no empiecen por *checkpoint* y se comprueba que sea compatible el modelo con el adaptador.

Ambos servicios se crean mediante funciones *factory*, que permiten recibir opcionalmente la ruta donde guardar los entrenamientos y la ruta del catálogo en el caso del servicio de ajuste fino, y la ruta donde buscar adaptadores en el caso del servicio de adaptadores. En caso de no especificar una ruta en concreto se usarán valores definidos por defecto. Para los entrenamientos se crea una carpeta por técnica y otra por entrenamiento, guardando dentro los diferentes puntos de control y el resultado final.

Con respecto a las técnicas de ajuste fino, ambas exponen los **hiperparámetros** *components*, lista con los componentes entrenables con la técnica para un modelo en concreto, *learning_rate*, *epochs*, *batch_size* para el número de muestras procesadas por *batch*, *gradient_accumulation_steps* para el número de *batches* a acumular antes de cada paso del optimizador y `max_grad_norm` para recorte de gradientes. Además, para el manejo de *checkpoints* de entrenamiento se cuenta con el descriptor *save_every_n_epochs*.

Para **LoRA**, se exponen como hiperparámetros concretos *rank* para el rango de las matrices LoRA y *alpha* para el factor de escalado de los pesos LoRA. Para las comprobaciones de compatibilidad con el modelo se comprueba que implemente el protocolo `SupportsLoRA` y para el *dataset* que sea `TextImageDatasetWrapper`. En el caso de **DreamBooth**, cuenta con los hiperparámetros específicos de *instance_prompt* para identificar el concepto concreto, *class_prompt* para describir la clase general, *number_of_class_images* para el número de imágenes de clase que generará el modelo y *prior_loss_weight* para el peso de la pérdida de preservación de la clase. Para las comprobaciones de compatibilidad con el modelo se comprueba que implemente el protocolo `SupportsDreamBooth` y para el *dataset* que sea `ImageDatasetWrapper`. Además, como detalle de implementación, DreamBooth cuenta con un tipo de *dataset* auxiliar para combinar las imágenes de instancia y las de clase, reutilizando las imágenes de forma circular en caso de disponer de cantidades diferentes de cada tipo.

Antes de empezar el entrenamiento, la técnica prepara el modelo diferenciando entre una ejecución nueva o una reanudación. En ambos casos se congelan inicialmente los componentes del pipeline y se habilitan solamente los parámetros correspondientes a los componentes seleccionados. Al reanudar, se comprueba la información de la metadata del *checkpoint* y se restauran la época y los estados del optimizador, *scheduler* y *scaler*. A continuación, se crea un optimizador `AdamW` y un `DataLoader` configurado con `collate_fn=lambda x: x` para evitar que PyTorch construya automáticamente los *batches* del *dataset*. De esta forma se mantiene su formato original y se delega la construcción en el método correspondiente del *wrapper*.

Durante el bucle de entrenamiento, se delega la transformación de las muestras en los tensores, el cálculo de la pérdida y la ejecución de la retropropagación en los métodos del *wrapper*, de forma que cada uno cuenta con sus especificaciones y el bucle de entrenamiento no depende de arquitecturas específicas. El proceso admite acumulación y recorte de gradientes, utilizando los valores proporcionados por el usuario, precisión mixta mediante `torch.autocast` y escalado de gradientes con `GradScaler` cuando se utiliza CUDA con FP16. LoRA utiliza un *scheduler* coseno con una fase inicial de calentamiento correspondiente al 10 % de las actualizaciones, mientras que DreamBooth utiliza un *scheduler* constante.

Con respecto a las diferencias entre ambas técnicas, LoRA inserta adaptadores mediante la librería PEFT y solamente se entrenan los parámetros insertados, manteniendo congelados los pesos originales del modelo. El resultado se guarda como adaptador de forma que se pueda cargar posteriormente sobre el modelo base. Por otra parte, DreamBooth entrena directamente los componentes seleccionados y se combinan las imágenes de instancia con las generales de clase. La función de pérdida suma la pérdida de instancia y la de preservación de clase, ponderada mediante `prior_loss_weight` y se guarda el modelo completo.

6.6.4 Pruebas

Para comprobar el comportamiento del módulo de ajuste fino se realizaron las siguientes pruebas unitarias, distinguiendo entre tres categorías. La mayor parte de las pruebas corresponden a pruebas de **funcionalidad** del sistema, donde se comprueba la obtención de las técnicas de ajuste fino compatibles dado un modelo y un *dataset*, la ejecución con LoRA, la reanudación de un entrenamiento y la carga de adaptadores, descarga y obtención de adaptadores compatibles de un modelo. Además, se comprueba la respuesta del sistema ante técnicas incompatibles con un modelo y un *dataset* determinados, identificador de entrenamiento ya

existente y reanudación no válida por incompatibilidades. Por otra parte, se realizan pruebas relativas a la **calidad**, donde se comprueba la mejoría de los entrenamientos sobre los modelos o adaptadores correspondientes. Por último, se realizan pruebas relativas a la **eficiencia** de los procesos de entrenamiento. Dichas pruebas se muestran en la Tabla 6.19:

Tabla 6.19: Resultados de test de la cuarta iteración.

CU	Nombre	Descripción	Resultado esperado	Resultado obtenido
CU-09	Obtención de técnicas compatibles (Funcionalidad)	Se solicita la lista de técnicas disponibles para un modelo y un *dataset* compatibles.	El sistema devuelve correctamente las técnicas de ajuste fino compatibles con ambos recursos.	Resultado esperado.
CU-09	Ejecución de ajuste fino con LoRA (Funcionalidad)	Se configura y ejecuta un entrenamiento LoRA con hiperparámetros válidos.	El sistema ejecuta el entrenamiento correctamente y genera el adaptador resultante en la ruta correspondiente, registrando además el entrenamiento en el catálogo.	Resultado esperado.
CU-09	Ejecución de ajuste fino con DreamBooth (Funcionalidad)	Se configura y ejecuta un entrenamiento DreamBooth con hiperparámetros válidos.	El sistema ejecuta el entrenamiento correctamente y genera el modelo resultante en la ruta correspondiente, registrando además el entrenamiento en el catálogo.	Resultado esperado.
CU-09	Tiempo de ejecución de los entrenamientos (Eficiencia)	Se mide el tiempo necesario para la ejecución de los procesos de entrenamiento de las dos técnicas.	Los tiempos son coherentes con la complejidad y características del modelo, *dataset*, técnica de entrenamiento e hiperparámetros correspondientes.	Resultado esperado.

. . . . . *(continúa en la página siguiente)* . . . . . .

Tabla 6.19 – *(viene de la página anterior)*

CU	Nombre	Descripción	Resultado esperado	Resultado obtenido
CU-09	Calidad de entrenamientos (Calidad)	Se comparan las generaciones producidas por el modelo antes y después del ajuste fino utilizando las mismas entradas.	Las generaciones obtenidas tras el entrenamiento reflejan una mejor adaptación al contenido del *dataset*, incorporando características, conceptos o estilos aprendidos durante el proceso de ajuste fino.	Resultado esperado.
CU-10	Reanudación de entrenamiento (Funcionalidad)	Se selecciona un entrenamiento cancelado o fallido con un punto de control válido.	El sistema recupera la técnica, los hiperparámetros y el último punto de control, y continúa el entrenamiento desde dicho estado.	Resultado esperado.
CU-07	Carga de adaptador entrenado (Funcionalidad)	Se carga el adaptador generado por un entrenamiento LoRA sobre el modelo base correspondiente.	El sistema carga correctamente el adaptador y permite utilizarlo durante la inferencia.	Resultado esperado.
CU-07	Obtención de adaptadores compatibles (Funcionalidad)	Se solicita la lista de adaptadores compatibles con un modelo concreto.	El sistema devuelve correctamente los adaptadores compatibles con el modelo. Resultado esperado.	
CU-07	Descarga de adaptadores (Funcionalidad)	Se solicita la descarga de los adaptadores cargados sobre un modelo.	El sistema descarga correctamente los adaptadores y restaura el comportamiento original del modelo.	Resultado esperado.
CU-09	Técnica incompatible (Funcionalidad)	Se solicita un entrenamiento con una técnica no compatible con el modelo o el *dataset* cargado.	El sistema rechaza la operación indicando que la técnica seleccionada no es compatible con los recursos cargados.	Resultado esperado.

. . . . . *(continúa en la página siguiente)* . . . . . .

Tabla 6.19 – *(viene de la página anterior)*

CU	Nombre	Descripción	Resultado esperado	Resultado obtenido
CU-09	Identificador de entrenamiento duplicado (Funcionalidad)	Se intenta iniciar un entrenamiento utilizando un identificador ya registrado en el catálogo.	El sistema rechaza la operación evitando sobrescribir el entrenamiento existente.	Resultado esperado.
CU-10	Reanudación no válida (Funcionalidad)	Se intenta reanudar un entrenamiento sin estado reanudable, sin punto de control válido o con recursos distintos a los registrados.	El sistema rechaza la operación indicando que el entrenamiento no puede reanudarse.	Resultado esperado.
CU-12	Eliminación de entrenamiento (Funcionalidad)	Se solicita eliminar un entrenamiento registrado junto con sus ficheros asociados.	El sistema elimina los ficheros del entrenamiento y borra su entrada del catálogo.	Resultado esperado.
CU-13	Limpieza del catálogo de entrenamientos (Funcionalidad)	Se solicita eliminar todos los entrenamientos registrados.	El sistema elimina los ficheros asociados a los entrenamientos y deja vacío el catálogo.	Resultado esperado.

6.7 Iteración 5: Interfaz gráfica

El objetivo de esta iteración es desarrollar la interfaz gráfica que permita al usuario hacer uso de las funcionalidades del sistema descritas a lo largo del capítulo de manera sencilla e intuitiva. Para ello, la interfaz debe proporcionar acceso a la gestión de modelos y *datasets*, realizar inferencias, aplicar operaciones de procesamiento sobre las entradas y salidas de la inferencia, realizar procesos de ajuste fino sobre modelos previamente cargados y la gestión de adaptadores y entrenamientos, cumpliendo de esta manera con los requisitos funcionales de RF1 a RF7. Además, esta iteración debe satisfacer el requisito no funcional RNF3, de forma que sea una interfaz gráfica sencilla e intuitiva para el uso del usuario. Para mantener la modularidad definida en RNF1, la interfaz se diseña de forma desacoplada del resto de módulos, delegando

su comportamiento al orquestador, que es quien se comunica con el resto de servicios.

6.7.1 Análisis

En el diagrama de casos de uso se dispone nuevamente del actor **Usuario**. A diferencia de las iteraciones anteriores, esta iteración no incorpora nuevos casos de uso, sino que proporciona interfaz gráfica a los ya definidos. De esta manera, la interfaz permite al usuario realizar las operaciones de gestión de recursos del sistema (Tablas 6.1 a 6.6), inferencia (Tabla 6.13), procesamiento 6.11, ajuste fino y gestión de entrenamientos (Tablas 6.14 a 6.18) desde una única aplicación.

6.7.2 Diseño

Para desacoplar la interfaz gráfica de la lógica de negocio del sistema, se diseñó la arquitectura mostrada en la Figura 6.15:

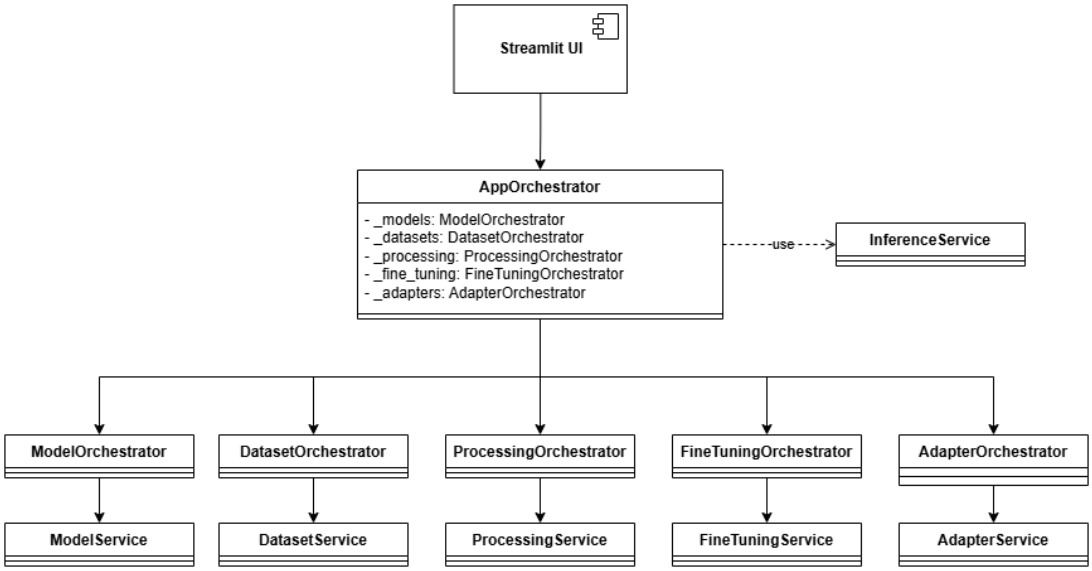


Figura 6.15: Diseño módulo de interfaz gráfica

Como se puede observar en la Figura 6.15, se distinguen tres componentes principales: la interfaz gráfica, el orquestador y los servicios. Para la comunicación entre la interfaz y el resto del sistema, se utiliza el patrón **Fachada** con el `AppOrchestrator` que actúa como único punto de entrada para la interfaz gráfica. De esta forma, la interfaz no conoce directamente los distintos servicios, sino que delega sus operaciones en el orquestador.

El **orquestador** se divide internamente en diferentes suborquestadores, cada uno encargado de gestionar las operaciones asociadas a un servicio concreto. Esta división permite

separar responsabilidades y evitar que el `AppOrchestrator` concentre toda la lógica de coordinación. Para el caso de la inferencia, al tratarse de un servicio con métodos estáticos y sin estado interno, se utiliza directamente desde el orquestador principal.

Con respecto al diseño de la interfaz gráfica, se organiza en tres vistas diferentes: carga de recursos, inferencia y ajuste fino. De esta manera, en la primera el usuario puede cargar los modelos y *datasets* que necesita, realizar inferencias sobre el modelo cargado en la segunda y, por último, realizar y gestionar entrenamientos en la tercera.

Los formularios de la interfaz se construyen de manera dinámica a partir de los descriptores proporcionados por el modelo para la inferencia, por los procesadores y por las técnicas de ajuste fino. Para cada valor a introducir por el usuario, se indica cuáles son los valores obligatorios, se muestra el valor por defecto para aquellos que disponen de uno y se proporciona una breve descripción del valor. Además, se indica un aviso al usuario sobre si un valor está fuera de los rangos establecidos o si falta un valor obligatorio, impidiendo poder ejecutar la operación hasta que se corrijan los errores.

Durante las operaciones largas, la interfaz deshabilita temporalmente controles y acciones de la interfaz gráfica de forma que mientras se carga un recurso, se ejecuta una inferencia o hay un entrenamiento activo, no se permite realizar otra operación. Además, se muestran indicadores de progreso, mensajes de error y notificaciones de éxito para informar al usuario.

Con este diseño se mantiene nuevamente la modularidad del sistema, al tener el módulo de la interfaz gráfica desacoplado de los módulos de carga de recursos, inferencia, procesamiento y ajuste fino gracias al orquestador, cumpliendo de esta manera el requisito RNF1 definido en la Sección 6.1.2.

6.7.3 Implementación

Con respecto a la implementación, **Streamlit** vuelve a ejecutar la aplicación tras cada interacción del usuario. De esta forma, para no perder el estado entre ejecuciones, se almacena en `st.session_state` la instancia de `AppOrchestrator`, junto con información relevante como el modelo cargado, el *dataset* activo, operación en curso, resultados de inferencia, estado del entrenamiento y errores.

Las operaciones de larga duración no se ejecutan al pulsar el botón. En su lugar, se registra la operación pendiente correspondiente en el estado de la sesión junto con su tipo y parámetros y se renderiza la aplicación. A continuación, la vista detecta la operación pen-

diente, muestra indicador de carga y guarda el resultado o el error antes de limpiar la operación.

La construcción de los formularios se centraliza en una función común que recibe un descriptor y genera el componente de Streamlit adecuado al tipo de dato: campo de texto, entrada numérica, selector, lista o casilla booleana. Además, esta misma función comprueba obligatoriedad, rangos y valores permitidos antes de habilitar la ejecución de la operación.

Los resultados de inferencia se representan según el tipo devuelto por el modelo, mostrando texto, imágenes, JSON o listas con el componente correspondiente de Streamlit. Los errores capturados se transforman en mensajes adaptados al usuario y se almacenan en el estado de sesión.

6.7.4 Pruebas

Para comprobar el correcto comportamiento de la interfaz gráfica se realizaron las pruebas funcionales que se recogen en la Tabla 6.20. En ellas se verifica que el usuario pueda realizar cada una de las operaciones definidas en el sistema desde la interfaz gráfica, así como la respuesta del sistema frente a la ejecución de operaciones de larga duración, validación de datos y errores.

Tabla 6.20: Resultados de test de la quinta iteración.

CU	Nombre	Descripción	Resultado esperado	Resultado obtenido
CU-01	Carga de recurso desde la interfaz	Se selecciona un modelo disponible y se solicita su carga desde la vista de recursos.	El modelo se carga correctamente y la interfaz actualiza el estado mostrando el recurso activo y el catálogo.	Resultado esperado.
CU-04	Desinstalación de recurso desde la interfaz	Se selecciona un recurso gestionado por el sistema del catálogo para desinstalar.	El sistema elimina de local el recurso y actualiza el estado del catálogo.	Resultado esperado.
CU-05	Eliminación de recurso desde la interfaz	Se selecciona un recurso por el sistema del catálogo para eliminar.	El sistema elimina de local el recurso si es gestionado por el sistema y elimina la entrada del catálogo.	Resultado esperado.

. . . . . *(continúa en la página siguiente)* . . . . . .

Tabla 6.20 – *(viene de la página anterior)*

CU	Nombre	Descripción	Resultado esperado	Resultado obtenido
CU-06	Limpieza de catálogo de recursos desde la interfaz	Se selecciona la operación desde la interfaz.	El sistema elimina de local todos los recursos gestionados por el sistema y vacía el catálogo.	Resultado esperado.
CU-07 y CU-08	Inferencia desde la interfaz	Se introducen los valores correspondientes para realizar la inferencia y se seleccionan opcionalmente los adaptadores y procesadores para las entradas y salidas.	La interfaz realiza la inferencia y muestra los resultados, además de las transformaciones aplicadas para las entradas y salidas.	Resultado esperado.
CU-09	Ajuste fino desde la interfaz	Se selecciona la técnica de ajuste fino y se introducen los hiperparámetros correspondientes.	La interfaz realiza el entrenamiento, mostrando *feedback* sobre el estado de ejecución.	Resultado esperado.
CU-10	Reanudación entrenamiento desde la interfaz	Se selecciona un entrenamiento reanudable del catálogo.	La interfaz reanuda el entrenamiento, mostrando *feedback* sobre el estado de ejecución.	Resultado esperado.
CU-12	Eliminación de entrenamiento desde la interfaz	Se selecciona un entrenamiento del catálogo para eliminar.	El sistema elimina los archivos asociados al entrenamiento de local y la entrada correspondiente del catálogo.	Resultado esperado.
CU-13	Limpieza de catálogo de entrenamientos desde la interfaz	Se selecciona la operación desde la interfaz.	El sistema elimina de local todos los archivos asociados a los entrenamientos y vacía el catálogo.	Resultado esperado.
CU-07 y CU-09	Validación de formularios	Se introducen valores fuera de rango o se omiten campos obligatorios.	La interfaz informa de los errores detectados e impide la ejecución de la operación hasta corregirlos.	Resultado esperado.

. . . . . *(continúa en la página siguiente)* . . . . . .

Tabla 6.20 – *(viene de la página anterior)*

CU	Nombre	Descripción	Resultado esperado	Resultado obtenido
CU-01, CU-07 y CU-09	Bloqueo durante operaciones largas	Se intenta iniciar una nueva operación mientras existe otra operación activa.	La interfaz deshabilita temporalmente las acciones disponibles e impide la ejecución simultánea de operaciones.	Resultado esperado.
RNF3	Gestión de errores	Se provoca un error durante la ejecución de una operación.	La interfaz muestra un mensaje de error sin interrumpir la ejecución de la aplicación.	Resultado esperado.

Los resultados obtenidos tras la realización de las pruebas mostradas en la Tabla 6.20 muestran el correcto funcionamiento de la interfaz gráfica.

Capítulo 7

Conclusiones

Una vez finalizado el proyecto, se han alcanzado todos los objetivos definidos inicialmente en la Sección 1.1. A lo largo del desarrollo se ha diseñado e implementado un sistema con interfaz gráfica que permite gestionar modelos y *datasets*, realizar inferencias, aplicar operaciones de procesamiento sobre los datos de entrada y salida, y ejecutar técnicas de ajuste fino, cumpliendo los requisitos funcionales y no funcionales descritos en las Secciones 6.1.1 y 6.1.2.

Como consecuencia de esto, el sistema desarrollado permite unificar el proceso de ajuste fino y uso de modelos generativos en una única aplicación con interfaz gráfica. Esto resulta de gran utilidad para simplificar el flujo de trabajo asociado al uso de modelos generativos, permitiendo utilizar diferentes tipos de modelos, arquitecturas y técnicas desde una misma aplicación. Además, su diseño modular y extensible facilita la futura incorporación de nuevas modalidades, modelos, técnicas de ajuste fino, procesadores y adaptadores de forma que el sistema pueda adaptarse a la evolución del campo.

Durante el desarrollo del sistema, la principal dificultad encontrada fue conseguir diseñar una arquitectura que abstraiga las particularidades de cada tipo de modelo y técnica de ajuste fino. El análisis de los distintos tipos de modelos, con sus grandes diferencias tanto en estructura como en utilización, puso de manifiesto la complejidad de lograr dicho objetivo. Actualmente el campo de los modelos generativos está en constante crecimiento con la aparición de nuevos modelos, modalidades y técnicas de ajuste fino. Esta evolución hace que las herramientas especializadas en arquitecturas o técnicas concretas puedan quedar limitadas ante la evolución de dicho campo. En este contexto, el sistema propuesto permite desacoplar y abstraer las particularidades de cada tipo de modelo y técnica de ajuste fino, facilitando la incorporación de nuevas arquitecturas, técnicas de ajuste fino, procesadores y adaptadores sin modificar el núcleo del sistema.

Sin embargo, el sistema presenta todavía ciertas limitaciones. Aunque ha sido diseñado para facilitar la incorporación de nuevas arquitecturas de modelos y técnicas, el número inicial de implementaciones es reducido. Además, el sistema es dependiente con respecto a los formatos utilizados por el ecosistema de Hugging Face. En el caso de los modelos, la identificación del tipo y la carga de recursos se apoyan en la estructura definida por las librerías utilizadas, así como en ficheros de configuración propios del ecosistema utilizados para reconocer el tipo de arquitectura o *pipeline* correspondiente. De esta manera, se presentan futuras mejoras al trabajo que serán tratadas en el Capítulo 8.

Como valoración personal, el proyecto ha permitido profundizar en áreas de especial interés que no se han podido abordar con el nivel de detalle deseado a lo largo de la carrera, como los modelos generativos y las técnicas de ajuste fino. Aunque se contaba previamente con conocimientos básicos sobre aprendizaje automático y algunas nociones de aprendizaje profundo y modelos de lenguaje, el análisis y la implementación de diferentes arquitecturas han permitido comprender la gran diversidad que existe en el campo de la inteligencia artificial generativa. Antes de comenzar con el proyecto, se conocían las diferencias existentes entre modalidades como texto, imagen o audio, pero tras el estudio de diferentes arquitecturas y técnicas de ajuste fino ha permitido comprobar que las diferencias son aún mayores, ya que cada tipo de modelo cuenta con su propia lógica de inferencia y de entrenamiento. De esta manera, tras la realización del proyecto se considera aún más necesario disponer de una aplicación que permita trabajar con los diferentes modelos generativos de forma unificada y centralizada.

Capítulo 8

Trabajo futuro

En este capítulo se tratan las diferentes líneas de trabajo a realizar sobre el sistema desarrollado. La base del proyecto, como se indica en el título, es lograr un sistema genérico y extensible, por lo que constituye un punto de partida para la unificación de una herramienta que permita trabajar con diferentes modelos generativos y técnicas de ajuste fino. De esta manera, las principales mejoras e incorporaciones a realizar en el sistema se detallan a continuación:

- **Fuentes de recursos**: El sistema cuenta actualmente con la obtención de recursos desde Hugging Face y desde almacenamiento local, pero está diseñado para permitir la incorporación de nuevas fuentes en el futuro, como por ejemplo CivitAI y ModelScope para modelos generativos o Kaggle para *datasets*, además de sistemas de almacenamiento remoto.

- **Modelos y Datasets**: El sistema proporciona actualmente soporte para modelos de difusión basados en arquitecturas como Stable Diffusion y Stable Diffusion XL, así como para *datasets* de tipo imagen y texto-imagen. Sin embargo, la arquitectura permite incorporar nuevas modalidades y arquitecturas como podrían ser modelos de lenguaje, modelos multimodales o modelos de audio o vídeo, así como nuevos tipos de *datasets* adaptados a dichas modalidades.

- **Técnicas de ajuste fino**: Al igual que con las fuentes de los recursos, los modelos y los *datasets*, el sistema permite incorporar nuevas implementaciones además de las incluidas inicialmente de LoRA y DreamBooth, como pueden ser QLoRA y Prefix Tuning, para modelos de lenguaje, o técnicas de ajuste fino completo adaptadas a las diferentes arquitecturas soportadas.

- **Procesadores**: Con respecto a los procesadores, se dispone inicialmente de un procesador para resumir texto y otro para modificar la resolución de imágenes. Con el diseño se facilita la incorporación de nuevos procesadores como podrían ser traducción de texto, detección de objetos en imágenes o transcripción de audio.

- **Adaptadores**: El soporte inicial del sistema para adaptadores durante la inferencia se limita a adaptadores LoRA, pero se pueden incorporar implementaciones nuevas para otros tipos de adaptadores como pueden ser LyCORIS, Textual Inversion o Hypernetworks, permitiendo ampliar las posibilidades de personalización de los modelos sin necesidad de almacenar modelos completos.
- **Gestión de entrenamientos**: El sistema permite registrar los entrenamientos realizados y almacenar información básica de los mismos. Como línea de trabajo futura, se podría incorporar la posibilidad de comparar entrenamientos, hiperparámetros y modelos resultantes, facilitando el análisis de resultados y la selección de mejores configuraciones adaptadas a la tarea específica. Además, podrían incorporarse mecanismos de visualización de métricas, seguimiento más detallado de los entrenamientos y validación.
- **Compatibilidad de formatos**: El sistema depende de los formatos esperados por las librerías del ecosistema Hugging Face utilizadas para la carga de modelos. Como línea de trabajo futura, se podría ampliar la compatibilidad incorporando nuevos métodos de carga y detección de formatos. De esta forma, el sistema podría trabajar con modelos que no tengan necesariamente la estructura definida por Hugging Face.

Apéndices

Lista de acrónimos

GPU *Graphics Processing Unit*. 8, 20, 50

IA Inteligencia Artificial. 1, 4, 8

LLM *Large Language Model*. 1, 2, 4, 13

LoRA *Low-Rank Adaptation*. 1, 2, 4, 6, 7, 9, 11, 12, 59, 65

PEFT *Parameter-Efficient Fine-Tuning*. 6, 7

Glosario

adaptador Conjunto reducido de parámetros entrenables que se carga sobre un modelo base para modificar o especializar su comportamiento sin necesidad de almacenar una copia completa del modelo ajustado. 7, 9, 11, 20, 25, 52, 53, 59, 64–69, 73, 75, 78

ajuste fino Técnica de entrenamiento que consiste en especializar un modelo genérico preentrenado en una tarea o dominio en concreto mediante el ajuste de sus parámetros con un dataset de menor tamaño. 1–4, 6–9, 11–18, 20, 22, 75–77

inferencia Proceso de ejecución de un modelo ya entrenado para obtener una salida a partir de una entrada proporcionada, sin modificar los parámetros del modelo. 2, 3, 8, 9, 11–18, 20, 76, 78

modelo generativo Modelo de inteligencia artificial capaz de generar datos como texto, imágenes, audio o vídeos a partir de diferentes patrones aprendidos durante la fase de entrenamiento. 1–4, 6, 8, 9, 11, 13–15, 17, 18, 20, 21, 25, 36, 75–77

Bibliografía

[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2021, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://aima.cs.berkeley.edu/

[2] Y. Cao, S. Li, Y. Liu, Z. Yan, Y. Dai, P. S. Yu, and L. Sun, "A comprehensive survey of AI-Generated content (AIGC): A history of generative AI from GAN to ChatGPT," 2023, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://arXiv.org/abs/2303.04226

[3] Gartner, Inc., "Gartner Forecasts Worldwide GenAI Spending to Reach $644 Billion in 2025," 2025, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://www.gartner.com/en/newsroom/press-releases/2025-03-31-gartner-forecasts-worldwide-genai-spending-to-reach-644-billion-in-2025

[4] OpenAI, "ChatGPT," 2026, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://chatgpt.com/

[5] ——, "OpenAI," 2026, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://openai.com/

[6] TechCrunch, "ChatGPT reaches 900 million weekly active users," 2026, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://techcrunch.com/2026/02/27/chatgpt-reaches-900m-weekly-active-users/

[7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," 2017, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://arxiv.org/abs/1706.03762

[8] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," 2020, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://arxiv.org/abs/2006.11239

[9] J. Kaplan, S. McCandlish, T. Henighan *et al.*, "Scaling Laws for Neural Language Models," 2020, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://arxiv.org/abs/2001.08361

[10] E. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," 2022, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://arxiv.org/abs/2106.09685

[11] Hugging Face, "Hugging Face Hub," 2026, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://huggingface.co

[12] ——, "Transformers Library," 2026, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://huggingface.co/docs/transformers

[13] ——, "Diffusers Library," 2026, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://huggingface.co/docs/diffusers

[14] ——, "PEFT Library," 2026, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://huggingface.co/docs/peft

[15] AUTOMATIC1111, "Stable Diffusion WebUI," 2026, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://github.com/AUTOMATIC1111/stable-diffusion-webui

[16] kohya-ss, "kohya_ss GUI for Stable Diffusion training," 2026, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://github.com/bmaltais/kohya_ss

[17] hiyouga, "LLaMA-Factory: Unified Efficient Fine-Tuning of LLMs," 2026, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://github.com/hiyouga/LLaMA-Factory

[18] Hugging Face, "AutoTrain," 2026, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://huggingface.co/autotrain

[19] N. Ruiz, Y. Li, V. Jampani, Y. Pritch, M. Rubinstein, and K. Aberman, "DreamBooth: Fine Tuning Text-to-Image Diffusion Models for Subject-Driven Generation," 2022, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://arxiv.org/abs/2208.12242

[20] A. Y. Ng and M. I. Jordan, "On Discriminative vs. Generative Classifiers: A Comparison of Logistic Regression and Naive Bayes," 2002, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://papers.nips.cc/paper/2001/file/7b7a53e239400a13bd6be6c91c4f6c4e-Paper.pdf

[21] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-Resolution Image Synthesis with Latent Diffusion Models," 2022, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://arxiv.org/abs/2112.10752

[22] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative Adversarial Nets," 2014, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://arxiv.org/abs/1406.2661

[23] D. Podell, Z. English, K. Lacey, A. Blattmann, T. Dockhorn, J. Muller, J. Penna, and R. Rombach, "SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis," 2023, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://arxiv.org/abs/2307.01952

[24] Python Software Foundation, "Python," 2026, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://www.python.org/

[25] PyTorch Foundation, "PyTorch," 2026, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://pytorch.org

[26] Hugging Face, "Datasets Library," 2026, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://huggingface.co/docs/datasets

[27] ——, "Accelerate," 2026, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://huggingface.co/docs/accelerate

[28] ——, "Safetensors," 2026, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://huggingface.co/docs/safetensors

[29] Pillow Contributors, "Pillow documentation - concepts and filters," 2025, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://pillow.readthedocs.io/en/stable/handbook/concepts.html#filters

[30] Streamlit, "Streamlit," 2026, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://streamlit.io

[31] ComfyUI, "ComfyUI," 2026, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://github.com/comfyanonymous/ComfyUI

[32] LM Studio, "LM Studio," 2026, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://lmstudio.ai

[33] Open WebUI, "Open WebUI," 2026, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://github.com/open-webui/open-webui

[34] Menlo Research, "Jan," 2026, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://jan.ai/

[35] Oobabooga, "text-generation-webui," 2026, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://github.com/oobabooga/text-generation-webui

[36] Ollama, "Ollama," 2026, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://ollama.com/

[37] Nerogar, "OneTrainer," 2026, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://github.com/Nerogar/OneTrainer

[38] OpenAccess AI Collective, "Axolotl," 2026, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://github.com/OpenAccess-AI-Collective/axolotl

[39] I. Sommerville, *Software Engineering*, 10th ed. Boston: Pearson, 2015, chapter 2: Software Processes.

[40] Glassdoor, "Glassdoor salaries," 2026, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://www.glassdoor.es/Sueldos/index.htm

[41] Hugging Face, "sshleifer/distilbart-cnn-12-6," 2026, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://huggingface.co/sshleifer/distilbart-cnn-12-6